

summary

- 1. 软件工程
- 2. 五十年代到00年代的特点
- 3. 团队
 - 3.1. 分类
 - 3.2. 团队建设
- 4. 质量保障
 - 4.1. 质量保障的措施
- 5. 配置管理
- 6. 需求
 - 6.1. 什么是需求
 - 6.2. 需求分层
 - 6.3. 需求分类
 - 6.4. 软件需求分类
- 7. 各类图
 - 7.1. 数据流图 非重点
 - 7.2. 实体关系图 不重要
 - 7.3. 用例图
 - 7.4. 概念类图(分析类图)
 - 7.5. 顺序图(交互图)
 - 7.6. 状态图
- 8. 为什么要建立需求规格说明?
- 9. 给定的用例, 指出错误, 并修正
- 10. 对给定的需求示例, 设计功能测试用例
- 11. 度量 不重要
 - 11.1. 功能点度量
- 12. 软件设计
 - 12.1. 软件设计的核心思想
 - 12.2. 软件设计的核心层次
- 13. 体系结构
 - 13.1. 体系结构概念
 - 13.2. 体系结构风格的优缺点
 - 13.3. 软件体系结构设计过程
 - 13.4. 包设计原则
 - 13.5. 了解的图片
 - 13.6. POJO和VO
 - 13.7. 体系中构建的之间的接口的定义 重要
 - 13.8. 集成的策略 不重要
 - 13.9. 体系结构开发集成测试用例
- 14. 人机交互设计
 - 14.1. 可用性
 - 14.2. 人机交互设计原则
 - 14.3. 精神模型
 - 14.4. 差异性

- 14.5. 导航
- 14.6. 反馈
- 14.7. 协作式设计
- 14.8. 小问题
- 15. 详细设计
 - 15.1. 详细设计的出发点
 - 15.2. 职责
 - 15.3. 职责分配
 - 15.3.1. GRASP原则(一般职责分配软件模式)
 - 15.4. 协作
 - 15.4.1. 协作的测试:Mock Object
 - 15.4.2. 集成测试
 - 15.5. 控制风格
 - 15.6. 部分题目
- 16. 内聚和耦合
 - 16.1. 结构化内聚
 - 16.2. 结构化耦合
 - 16.3. 面向对象耦合:
 - 16.4. 这部分区分需要解释
- 17. KWIC的实现
- 18. 信息隐藏
 - 18.1. 基本思想
 - 18.2. 秘密
 - 18.3. 两种常见的信息隐藏
- 19. 设计原则汇总
 - 19.1. 内聚耦合度量(了解)
- 20. 封装
 - 20.1. 封装实现的细节
- 21. 可修改性、可扩展性、灵活性
- 22. 几个设计模式
 - 22.1. 策略模式
 - 22.2. 抽象工厂模式
 - 22.3. 迭代器模式
 - 22.4. 单件模式
- 23. 软件构造
- 24. 代码改进
- 25. 测试
 - 25.1. 单元测试构造
- 26. 契约式编程
- 27. 防御式编程
- 28. 表驱动
- 29. 代码复杂度度量
- 30. 白盒测试和黑盒测试
 - 30.1. 黑盒测试
 - 30.1.1. 黑盒测试方法-等价类划分
 - 30.1.2. 黑盒测试方法-边界值分析

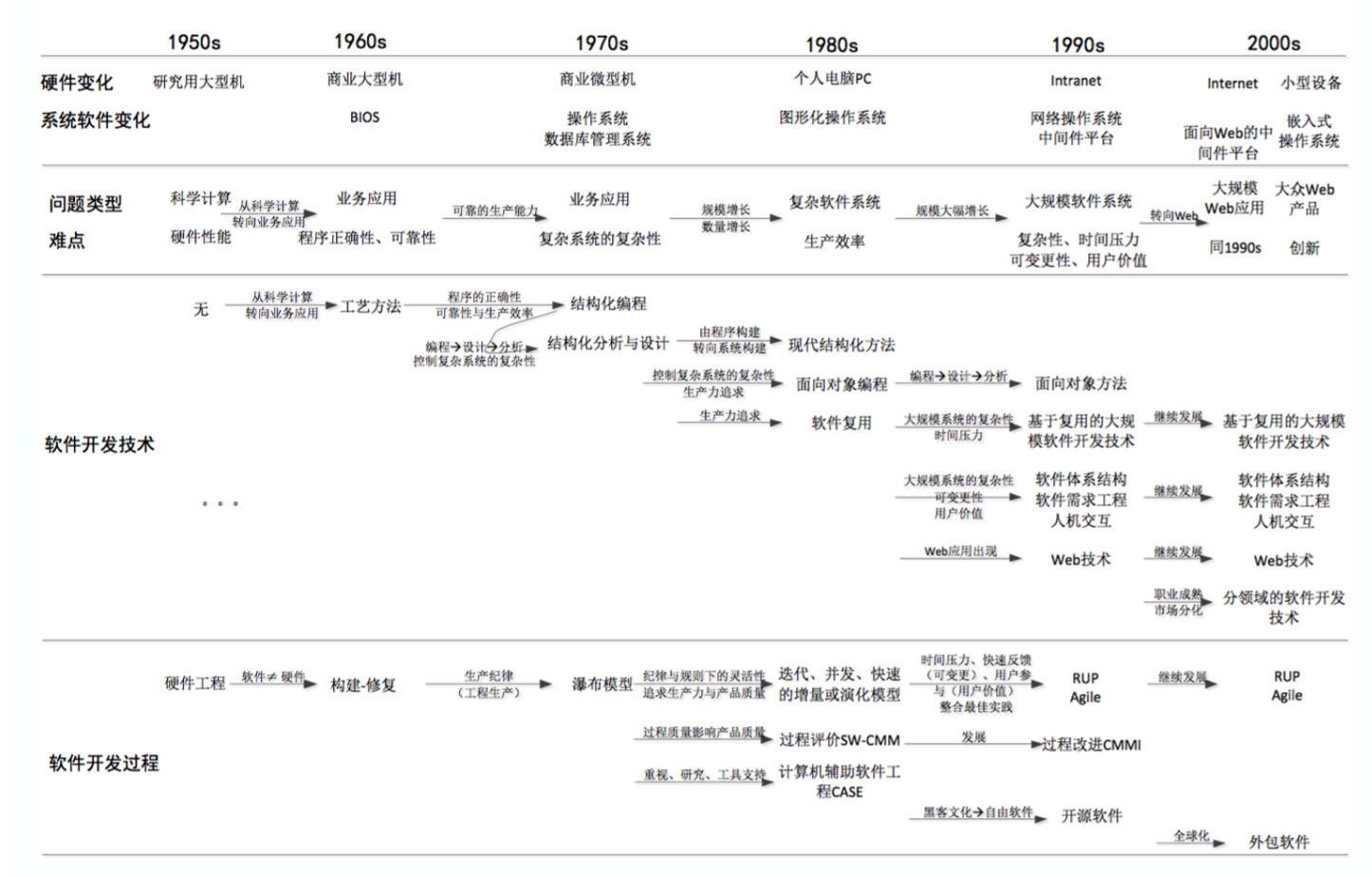
- 30.1.3. 黑盒测试方法-决策表
 - 30.1.4. 黑盒测试方法-状态转换
- 30.2. 白盒测试
 - 30.2.1. 白盒测试方法-语句覆盖
 - 30.2.2. 白盒测试方法-条件覆盖
 - 30.2.3. 白盒测试方法-路径覆盖
- 31. 测试
- 32. 软件维护
 - 32.1. 软件维护的重要性
 - 32.2. 开发可维护软件的方法
 - 32.3. 软件维护的过程
 - 32.4. 用户文档和系统文档
 - 32.4.1. 用户文档
 - 32.4.2. 用户文档的要素
 - 32.4.3. 系统文档
 - 32.5. 软件演化生命周期模型
 - 32.5.1. 步骤
 - 32.5.2. 优缺点
 - 32.5.3. 实际模型
 - 32.6. 逆向工程
 - 32.7. 再工程
- 33. 部分题目
- 34. 说明以下需求属于哪种类型
- 35. 判断需求的正确性或者问题

1. 软件工程

1. 应用系统的、规范的、可量化的方法来开发、运行和维护软件，即将工程应用到软件。
2. 对1)中各种方法的研究。

2. 五十年代到00年代的特点

1. 1950s: 科学计算; 以机器为中心进行编程; 像生产硬件一样生产软件。
2. 1960s: 业务应用(批量数据处理和事物计算); 软件不同于硬件; 用软件工艺的方式生产软件。
3. 1970s: 结构化方法; 瀑布模型; 强调规则和纪律。它们奠定了软件工程的基础, 是后续年代软件工程发展的支撑。
4. 1980s: 追求生产力最大化; 现代结构化方法/面向对象编程广泛应用; 重视过程的作用。
5. 1990s: 企业为中心的大规模软件系统开发; 追求快速开发、可变更性和用户价值; web应用出现
6. 2000s: 大规模web应用; 大量面向大众的web产品; 追求快速开发、可变更性、用户价值和创新。



3. 团队

3.1. 分类

- 1. 主程序员团队
- 2. 民主团队
- 3. 开放团队

3.2. 团队建设

- 1. 建立团队章程
- 2. 持续成功
- 3. 和谐沟通
- 4. 不断总结
- 5. 避免团队杀手
 - i. 防范式管理
 - ii. 官僚主义
 - iii. 地理分布：异地办公
 - iv. 时间分割：保证全天候有人在
 - v. 产品质量的降低
 - vi. 虚假的最后期限
 - vii. 小圈子管理

4. 质量保障

4.1. 质量保障的措施

1. 需求开发：需求评审和需求度量；
2. 体系结构：体系结构评审和集成测试(持续集成)；
3. 详细设计：详细设计评审、设计度量和集成测试(持续集成)；
4. 构造阶段：代码评审、代码度量、测试(测试驱动和持续集成)；
5. 测试阶段：测试、测试度量。
6. 要及时的根据保障计划进行质量验证，质量验证的方法主要有评审、测试和质量度量三种。

5. 配置管理

1. 标识配置项:确定应该被保留的部分，并且给予他们确定标识，包含配置项的特征，包括生产者、基线建立时间、使用者等。
2. 版本管理：极其重要
3. 变更控制：变更请求表单，教材61页
4. 配置审计:验证配置项的完整性、正确性、一致性和可追踪性。
5. 状态报告:反映当前的配置状态。
6. 软件发布管理:将配置项发布到开发活动之外，例如发布给客户。

6. 需求

6.1. 什么是需求

1. 用户为了解决问题或达到某些目标所需要的条件或能力；
2. 系统或系统部件为了满足合同、标准、规范或其它正式文档所规定的要求而需要具备的条件或能力；
3. 对1或2中的一个条件或一种能力的一种文档化表述。

6.2. 需求分层

1. 业务需求：
 - i. 业务需求是高层次的解决方案和系统特性、系统开发的战略出发点、高层次的需求，描述为什么要开发系统。
 - ii. Eg.在系统使用3个月后，销售额度应该提高20%(期望，没有从软件角度进行描述，业务需求)
2. 用户需求：
 - i. 执行具体任务的用户对系统所能完成任务的期望，描述了系统能帮用户做什么(直接用户、间接用户)
 - ii. Eg.在系统要帮助收银员完成销售处理
3. 系统级需求
 - i. 需求分析模型：用户对**系统行为的期望**，每个系统级需求反映了一次外界与系统的交互行为，或者系统的一个**实现细节**(和用户需求有着很大的区别)

- ii. Eg.在接到客户经理的请求后,系统应该为客户经理提供所有会员的个人信息。

6.3. 需求分类

1. 需求

- i. 项目需求(人的数量、计划成本、时间)
 - R5: 项目的成本要控制在60万元人民币以下。
 - R6: 项目要在6个月内完成。
- ii. 过程需求(人的分工、合作、方法、工具)
 - R7: 在开发中,开发者要提交软件需求规格说明文档、设计描述文档和测试报告。
 - R8: 项目要使用持续集成方法进行开发。
- iii. 系统需求
 - a. 软件需求
 - b. 硬件需求
 - c. 其他需求
- iv. 其他需求
 - R9: 系统要购买专用服务器,其规格不低于....。
 - R10: 系统投入使用时,需要对用户进行1个星期的集中培训。

2. 不切实际的需求

- R11: 系统要分析会员的购买记录,预测该会员将来一周和一个月内、会购买的商品;(技术上不可行)
- R12: 系统要能够对每月的出入库以及销售行为进行标准的财务分析;(在有限的资源条件下可行)
- R13: 在使用系统时,收银员必须要在2个小时内完成一个销售处理的所有操作。(超出了软件所影响的问题域范围)

6.4. 软件需求分类

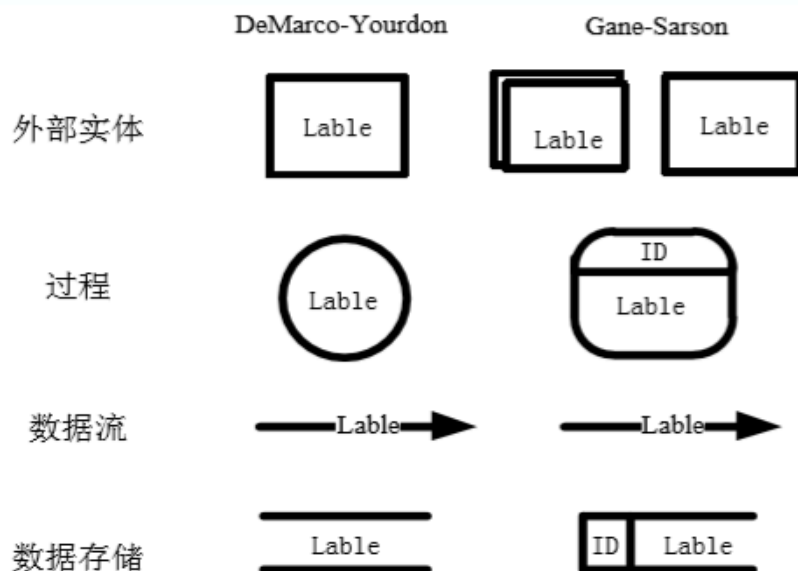
- 1. 功能需求: 和系统主要共作相关的需求,即在不考虑物理约束的情况下,用户希望系统所能够执行的活动,这些活动可以帮助用户完成任务。**功能需求主要表现为系统和环境之间的行为交互。**
 - i. Eg.在接到客户经理的请求后,系统应该为客户经理提供所有会员的个人信息。
- 2. 数据需求(属于功能需求的一种,DR): **功能需求**的补充: 如果在功能需求部分明确定义了**相关的数据结构**,那么就不需要再行定义数据需求。数据需求是需要在数据库、文件或者其他介质中存储的数据描述,通常包括下列内容:
 - 各个功能使用的**数据信息**;
 - 使用频率;
 - 可访问性要求;
 - **数据实体及其关系**;
 - 完整性约束;
 - **数据保持**要求。
 - Eg.系统需要存储的数据实体及其关系为图6-14的内容。(数据实体及其关系)
 - Eg.系统需要存储1年内的销售记录和退货记录。(数据保持)
- 3. 性能需求: 系统整体或系统组成部分应该拥有的性能特征,例如CPU使用率、内存使用率等。
 - i. 速度: 系统完成任务的时间(所有用户查询必须在10s内完成)
 - ii. 容量: 系统能存储的数据量(系统因该能够存储至少100万个销售信息)
 - iii. 吞吐量: 系统在连续的时间内完成的事务数量(解释器每分钟应该能够至少解析5000条没有错误的语句)
 - iv. 负载: 系统可以承载的并发工作量(系统应该允许50个营业服务器同时从集中服务器上进行数据的上传或下载)

- v. 实时性: 严格的实时要求(系统监测到异常时, 监视器必须在0.5s内发出警报, 和故障警报不一样, 故障不是系统的正常功能)
- 4. 质量需求(QA): 系统为了满足**规定的及隐含**的所有要求而需要具备的要素称为质量
 - i. 可靠性: 在规格时间间隔内和规定条件下, 系统或部件执行所要求能力的的能力。(在进行数据的下载和上传中, 如果网络故障, 系统不能出现故障。能不能检测网络中断, 并且进行恢复)
 - ii. 可用性: 软件系统在投入使用时可操作和可访问的程度或能实现其指定系统功能的概率。(系统的可用性要达到98%)
 - iii. 安全性: 软件组织对其程序和数据进行未授权访问的能力, 未授权的访问可能是有意, 也可能是无意的。(VIP顾客只能查看自己的个人信息和购买记录)
 - iv. 可维护性: 软件系统或部件能修改以排除故障、改进性能或其他属性或适应变更了的环境的容易程度, 包括可修改性(Modifiability)和可扩展性(Extensibility)。(如果系统要增加新的特价类型, 要能够在2个人月内完成。)
 - v. 可移植性: 系统或部件能从一种硬件或软件环境转换至另外一种环境的特性。(集中服务器要能够在1人月内从Window 7操作系统更换到Solaris 10操作系统。)
 - vi. 易用性: 与用户使用软件所花费的努力及其对使用的评价相关的特性。(使用系统1个月的收银员进行销售处理的效率要达到10件商品/分钟。)
 - vii. 往往会有形容词和副词
- 5. 对外接口: 系统和环境中其他系统之间需要建立的接口, 包括硬件接口、软件接口、数据库接口等等。
 - i. 接口的用途
 - ii. 接口的输入输出
 - iii. 数据格式
 - iv. 命令格式
 - v. 异常处理要求
 - vi. Eg. 注册使用Google Maps API
- 6. 约束: 进行系统构造时需要遵守的约束, 例如**编程语言、硬件设施**等
 - i. 系统开发及运行的环境(包括目标机器、操作系统、网络环境、编程语言、数据库管理系统等,Eg.系统使用Java语言进行开发)
 - ii. 问题域内的相关标准(包括法律法规、行业协定、企业规章等。)
 - iii. 商业规则:(用户在任务执行中的一些潜在规则也会限制开发人员设计和构建系统的选择范围)
 - iv. Eg. 已过保质期的食品不能销售
 - v. Eg. 顾客可以使用美元付款
- 7. 注:需求的灵活性

7. 各类图

7.1. 数据流图 非重点

- 1. 图例:
 - i. 外部实体(长方形)
 - ii. 过程(圆形)
 - iii. 数据流(实心箭头)
 - iv. 数据存储

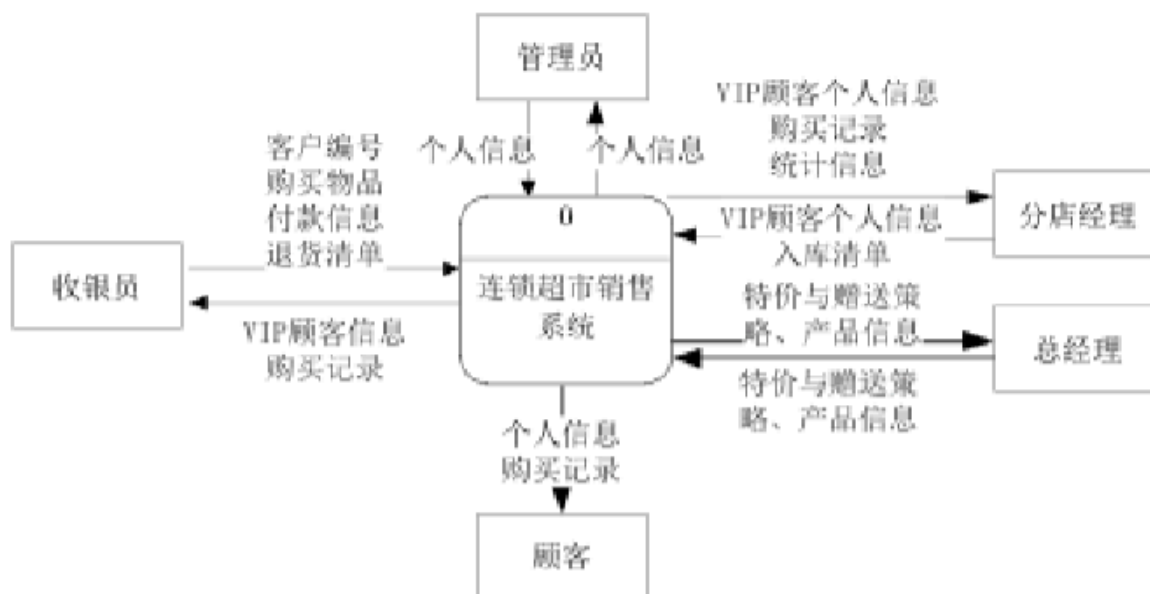


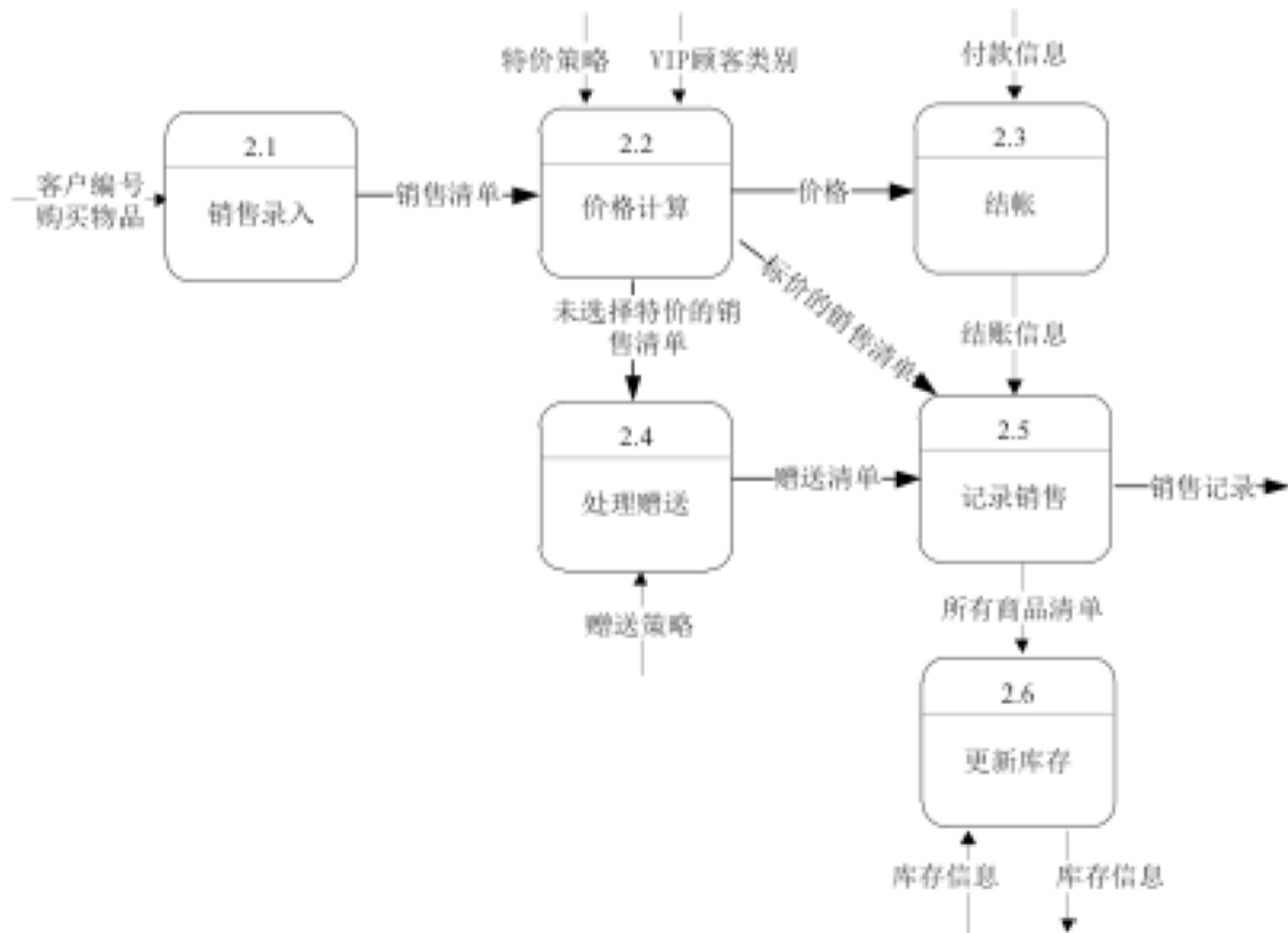
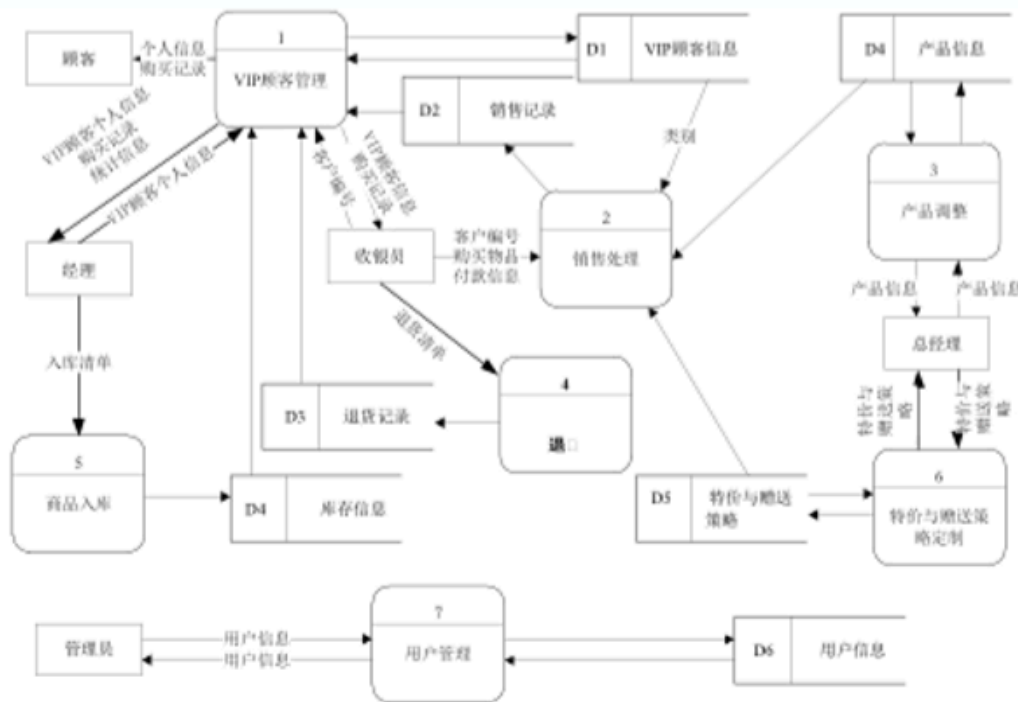
Data Flow Diagram

— — Flow Modeling Notation (基本元素)

2. 上下文图、0层图和N层图

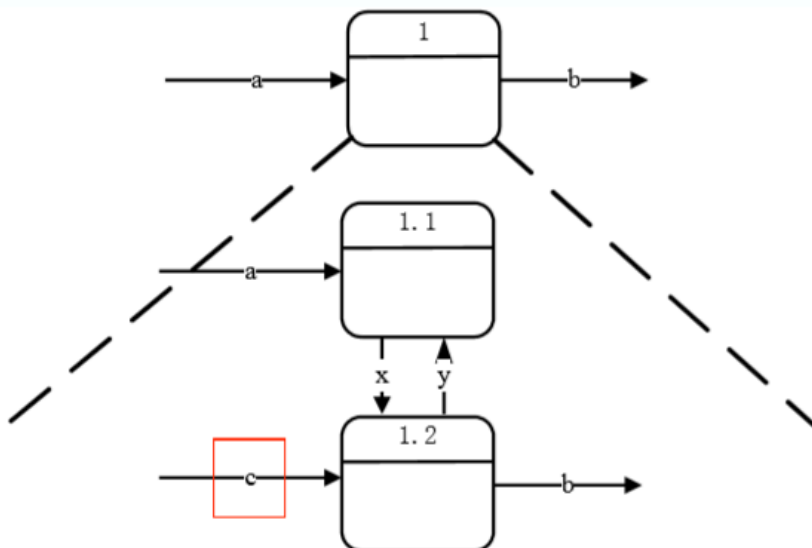
- 上下文图是DFD的最高层次的图，是系统功能的最高抽象。上下文将整个系统看做一个过程，这个过程实现系统的所有功能。
- 0层图通常被用作整个系统的功能概图。
- 1层图分析以确定下一级转换
- N层图不断降解





1层图

3. 过程分解的平衡原则



过程分解的平衡原则

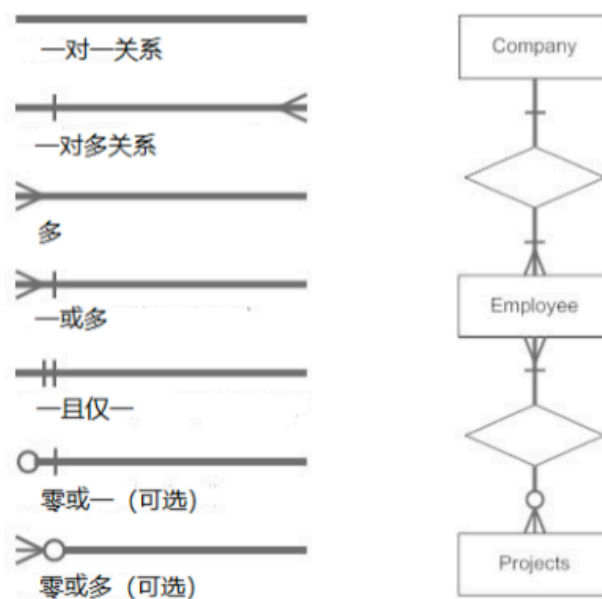
父过程的输入流为 a,父过程输出流为 b,子图 的输入流为 a 和 c,子图的输出流为 b。这样,它们的输入流存在差异,破坏了平衡性

7.2. 实体关系图 不重要

1. 图例:

- i. 传统实体:实体是需要在系统中收集和存储的现实世界事物的类别描述
- ii. 属性:数据对象包含的一组作为对象的方面、质量、特征或描述符的属性
- iii. 关系:传统实体之间的关系

OWN



Cardinality and Multiplicity

2. 建立步骤

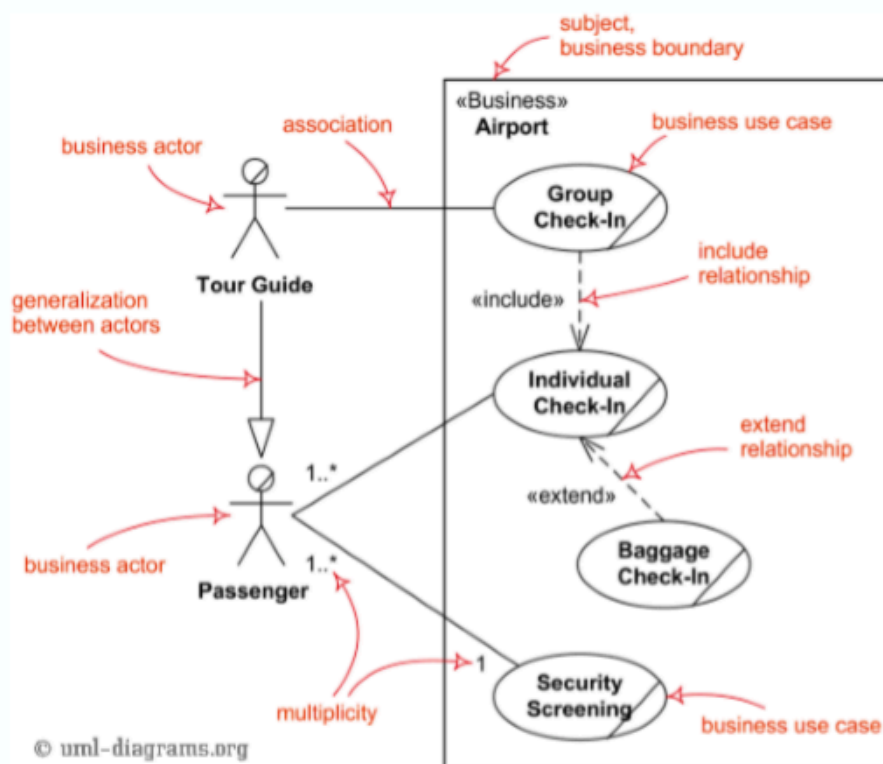
- i. 对所有数据对象(实体)及其相互之间的"连接"建模
- ii. 对所有实体和关系建模
- iii. 对所有实体、关系和属性建模, 以提供进一步的深度

7.3. 用例图

1. 用例: 用例是"在系统(或者子系统或者类)和外部对象的**交互**当中所执行的行为序列的描述, 包括**各种不同的序列和错误的序列**, 它们能够联合提供一种**有价值的服务**"[Rumbaugh2004]。

2. 图例:

- i. 用例: 椭圆
- ii. 参与者: 小人, 是系统或其他系统对要开发的系统所扮演的角色。
- iii. 关系: 简单的就是一条直线, 包括相关、泛化关系、包含关系和继承关系。
- iv. 系统边界: 是一个框



Relationship

3. 用例图的建立步骤

- i. 目标分析与解决方向的确定: 需要实现什么, 属于需求
- ii. 寻找参与者:
- iii. 寻找用例:
- iv. 细化用例: 判断标准是用例描述了为应对一个业务事件, 由一个用户发起, 并在一个连续时间段内完成, 可以增加业务价值的任务。

4. 注意:

- i. 不要将用例细化为没有独立业务价值的单个操作: 例如, 不要将用户管理细化为增加、修改和删除三个更小的用例, 因为它们要联合起来才能体现出业务价值。
- ii. 不要将同一个业务目标细化为不同用例: 例如 **特价策略制定** 和 **赠送策略制定**。
- iii. 不要将没有业务价值(而是技术实现需要)的内容作为用例: 常见的错误有 **登录** (应该描述为安全性质量需求)、"数据验证/输入/输出数据检查" (应该描述为数据需求或者业务规则)、"连接数据库" (属性软件内部实现而不是需求)、网络传输等。
- iv. 不要将单个步骤细化为用例
- v. 不要将片面的一个方面细化为用例

- 为下列描述建立用例模型（6分），要求明确给出建模过程（4分）。
- 现在需要开发一个简化了的大学图书馆系统，它有几种类型的借书人，包括教职工借书人、研究生借书人和本科生借书人等。借书人的基本信息包括姓名、地址和电话号码等。对于教职工借书人，还要包括诸如办公室地址和电话等信息。对于研究生借书人，还要包括研究项目和导师信息等。对于本科生借书人，还要包括项目和所有学分信息等。
- 图书馆系统要跟踪借出书本信息。当一个借书人捧着一堆书去借书台办理借书手续时，借出这个事件就发生了。随着时间的过去，一个借书人可以多次从图书馆中借书。一次可以借出多本图书。
- 如果借书人想要的书已被借出，他可以预约。每个预约只针对一个借书人和一个标题。预约日期、优先权和完成日期等信息需要维护。当借书完成，系统会将这本书与借出联系起来。
- 借书人根据图书馆的信息来检索书名，同时检索这本书是否可以被借出。如果一本书的所有副本都被借出了，那么借书人可以根据书名预订这本书。当借书人把书拿到借书台的时候，管理员可以为这些书办理归还手续。管理员要跟踪新书到达的情况。
- 图书馆的管理者有属于自己的活动。他们要分类打出关于书的标题的表格，还要在线检查所有过期未还的书，也标出来。而且，图书馆系统还可以从另外一个大学的数据库中访问和下载借书人的信息。

7.4. 概念类图(分析类图)

1. 图例:

i. 对象

- 标识符:对象自治、对象请求写作
- 状态:存储数据，如密码、名称
- 行为:利用数据做什么

ii. 类:对象集合的抽象

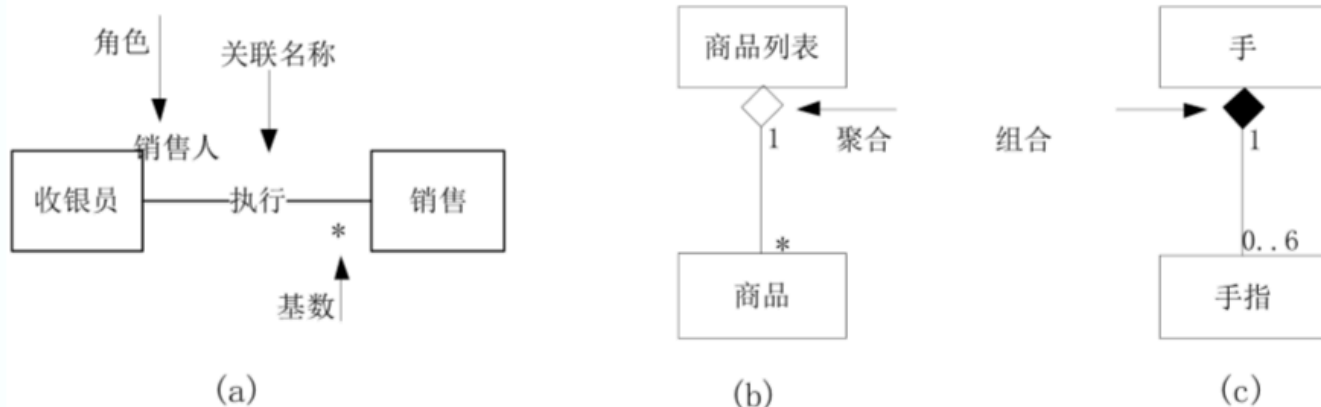
iii. 链接(link)(dependency)

- 对象之间的互相协作的关系
- 描述了对象之间的物理或业务联系

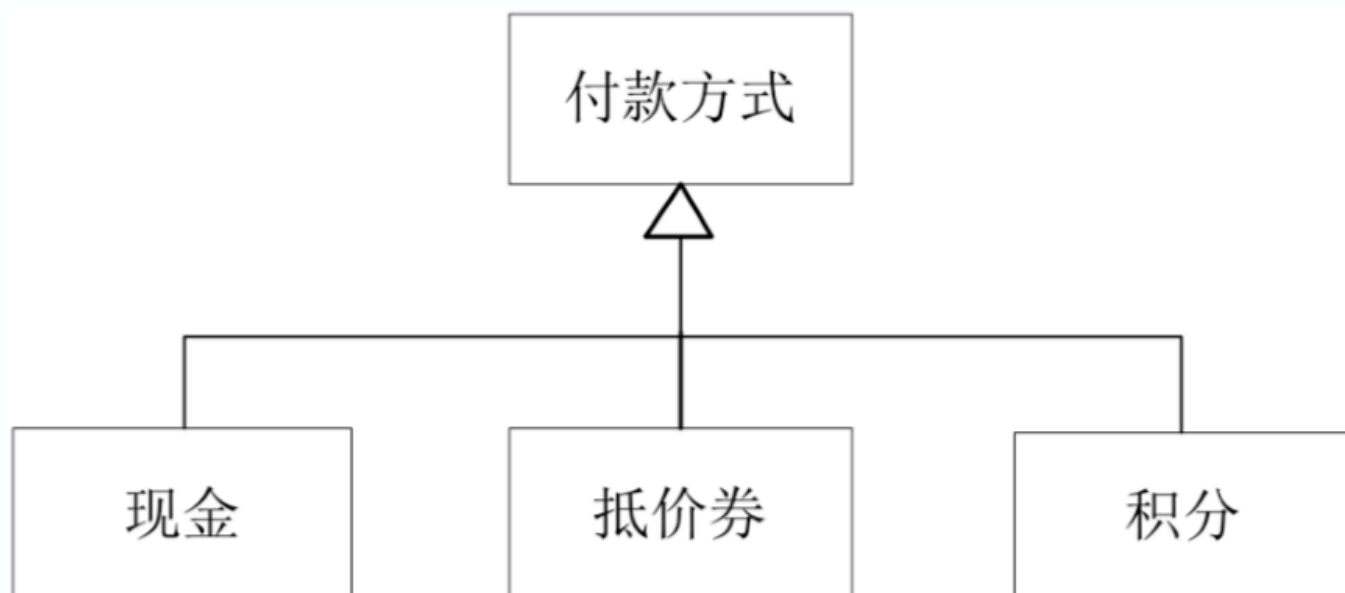
iv. 关联

- 对象之间链接的抽象
- 聚合与组合，聚合集合可以为空

v. 继承:泛化关系



案例



案例

2. 建立步骤

i. 对每个用例文本描述，尤其是场景描述，建立**局部**的概念类图

- 根据用例的文本描述，**识别候选类**(名词分析法)
- **筛选候选类**，确定概念类(状态和行为)
 - 状态和行为:概念类
 - 状态:其他概念类的行为
 - 行为:需求是否遗漏
 - 无状态无行为:完全剔除
- **识别关联**
- **识别重要属性**

ii. 将所有用例产生的局部概念类图进行合并，建立软件系统的整体概念类图

iii. 自己注:先画关联关系，再添加类的属性

3. 候选类识别(行为 + 状态)

i. 行为分析

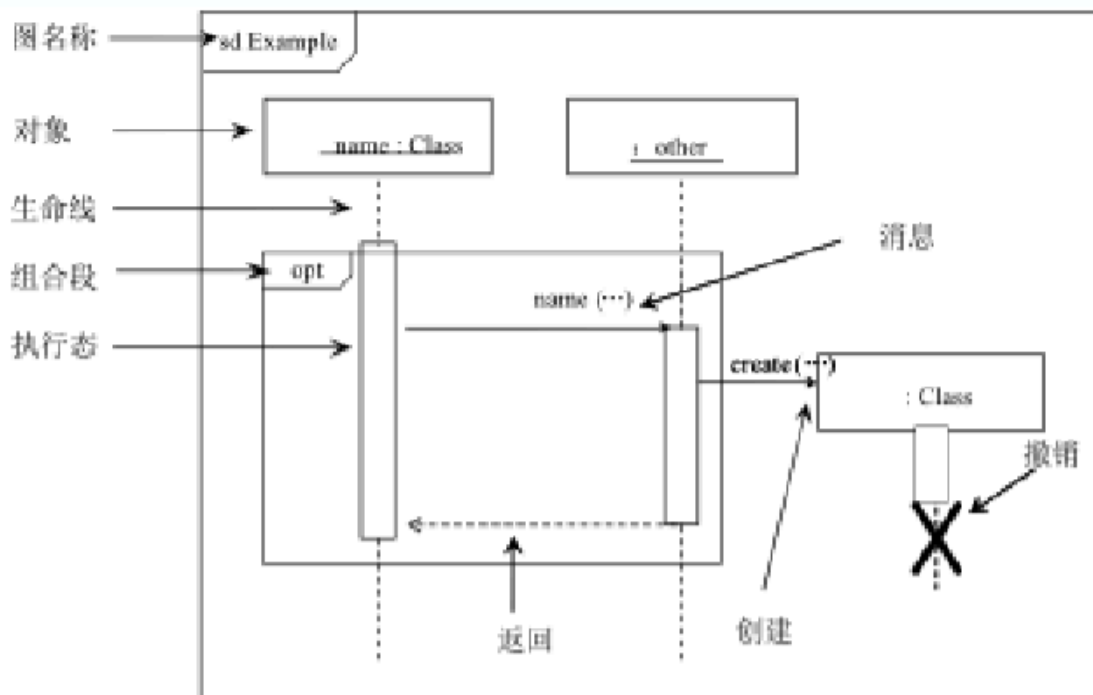
ii. 名词分析

iii. CRC

- 下面是一段需求描述，请依据其建立ATM机系统的领域类图（即分析类图）。
- A银行计划在B大学开设银行分部，计划使用ATM机提供全部服务。ATM系统将通过显示屏幕、输入键盘（有数字键和特殊符号键）、银行卡读卡器、存款插槽、收据打印机等设备与客户交互。客户可以使用ATM机进行存款、取款、余额查询等操作，它们对帐户的更新将交由账户系统的一个接口来处理。安全系统将为每个客户分配一个PIN码和安全级别。每次事务执行之前都需要验证该PIN码。在将来，银行还计划使用ATM机支持一些常规的操作，例如地址和电话号码修改。

7.5. 顺序图(交互图)

1. 图例：



一个简单示例



同步消息



异步消息



返回消息

消息种类

2. 系统顺序图图例:

- alt一定要选(多选一):注意, 每一种可选分支之间要用虚线分割, 而且在表示执行态的圆柱上面要写监护条件, 放在[]里面。
- opt一定要选(选择0或者1)
- loop:表示循环, 在旁边使用[]书写循环条件

系统顺序图

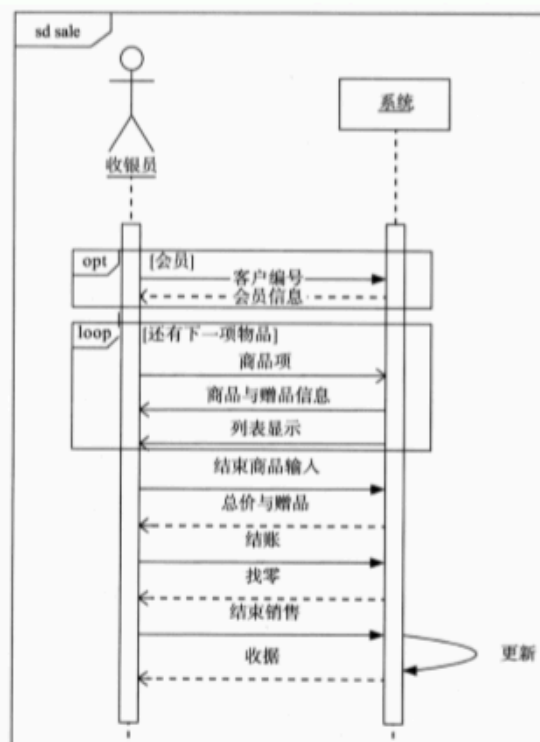
同步消息是实现, 三角箭头。

返回消息应该是虚线

Opt 是可选项

loop 是循环

alt 多选一



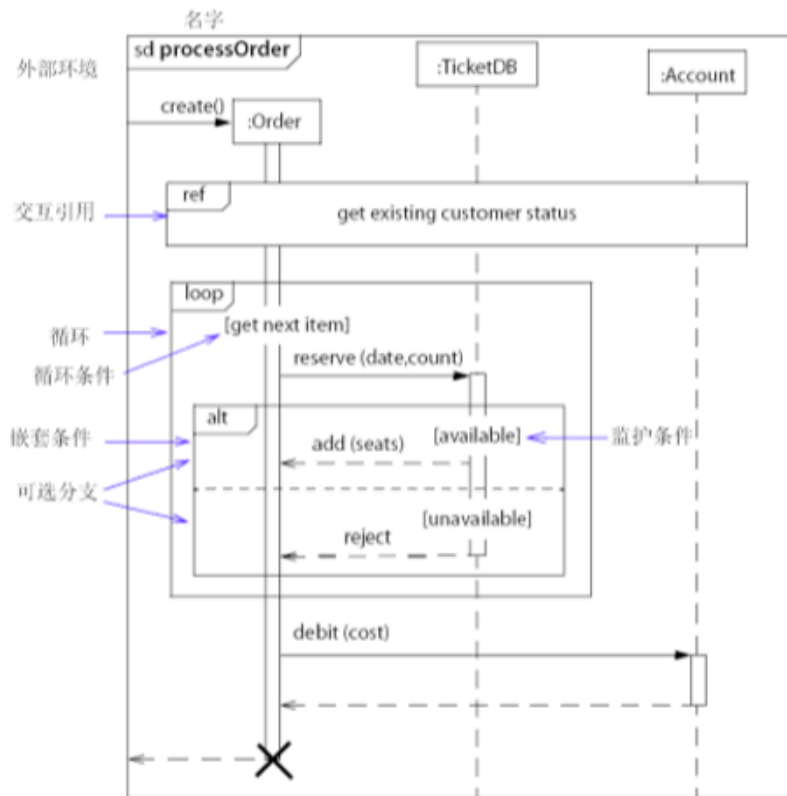


图 14-20、特化图示示例，源自 [Rumbaugh2004]

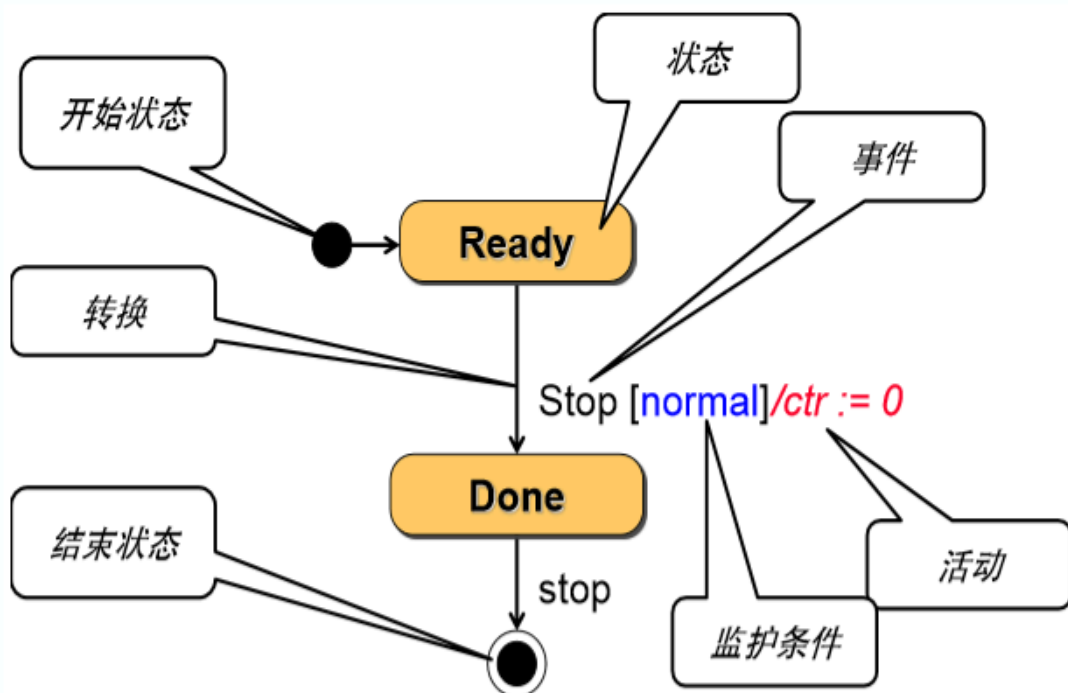
3. 步骤:

- i. 确定上下文环境
- ii. 根据用例描述找到交互对象
- iii. 按照用例描述中的流程顺序逐步添加消息s

7.6. 状态图

1. 图例:

- i. 状态：一组可观察的情况，描述了一个系统在给定时间的行为
- ii. 状态转换：从一个状态到另一个状态的转换
- iii. 事件：使系统表现出某种可预测的行为形式的事件
- iv. 行为：由于过渡而发生的过程



示例

2. 步骤:

- 确定上下文环境，明确状态主体和状态主题对应的上下文环境
- 识别状态：状态的主体表现出的一些稳定状态
- 建立状态转换：建立状态之间的转换。
- 补充详细信息，完善状态图

8. 为什么要建立需求规格说明？

- 方便交流：**软件开发过程中，**子任务与人员**之间存在错综复杂的关系，存在大量的沟通和交流，所以要编写软件开发中要编写不同类型的文档，每种文档都是针对项目中需要广泛交流的内容。因为**软件需求需要进行广泛交流**，所以要把需求文档化。
- 跟踪和度量：**需求规格说明是在**软件产品的角度以系统级需求列表的方式**描述软件系统解决方案，书写需求规格说明，可以建立管理控制的基线，方便任务分配，制定工作计划，进行跟踪和度量。
- 过程管理：**在实验中，需求规格的重要性不只体现在结果上，还包括中间过程，在书写需求规格过程中，才真正把问题域的问题和分析模型的成果转化为系统级需求，方便小组成员真正明确需求，个人认为在这个阶段包含一部分的需求在发现和完整化。

9. 给定的用例，指出错误，并修正

- 技术文档写作要点(简洁，精确，易读，易修改);
 - 简洁:动词名词+辅助词，不要使用复杂长句、形容词和副词。
 - 精确:不能产生歧义或无法理解。

- iii. 易读(查询): 有效使用引言、目录、索引等能够增强文档易读性的方法, 使用系统化的方式组织内容信息, 提供文档内容的可读性。
 - iv. 易修改: 使用相同的语句格式组织相关联或相似的信息; 使用列表组织独立、并列的信息; 使用编号表达繁杂信息之间的关系。引用而不是重复
2. 需求书写要点(使用用户术语, 可验证, 可行性);
3. 需求规格说明文档书写要点(充分利用标准的文档模板, 保持所以内容位置得当; 保持文档内的需求集具有完备性和一致性; 为需求划分优先级)

歧义词汇	改进方法
可接受的、足够的	具体定义可接受的内容, 说明系统怎样判断“可接受”或“足够”
依赖	描述依赖的原因, 数据依赖? 服务依赖? 还是资源依赖? 等等
有效的	明确“有效”所意味的具体实际情况
快的、迅速的	明确指定系统在时间或速度上可接受的最小值
灵活的	描述系统为了响应条件变化或需求变化而可能发生的变更方式
改进的、更好的、优越的	定量说明在一个专门的功能领域内, 充分改进的程度和效果
包括但不限于、等等、诸如	应该列举所有的可能性, 否则就无法进行设计和测试
最大化、最小化、最优	说明对某些参数所能接受的最大值和最小值
一般情况下、理想情况下	需要增加描述系统在异常和非理想情况下的行为
可选择地	具体说明是系统选择、用户选择还是开发人员选择
合理的、必要的、适当	明确怎样判断合理、必要和适当
健壮的	显式定义系统如何处理异常和如何响应预料之外的操作
不应该	试着以肯定的方式陈述需求, 描述系统应该做什么
最新技术水平的	定义其具体含义, 即“最新技术水平”意味什么
充分的	说明“充分”具体包括哪些内容
支持、允许	精确地定义系统的功能, 这些功能组合起来支持某些能力
用户友好的、简单的、容易的	描述系统特性, 用这些特性说明词汇所代表的用户期望的实质

1. "After the payment process is complete, the relevant information should be appended to a log file."模糊的

2. "The system should be constructed so that it will be easy to add new functionality in the future."(b)模糊的

3. "The price of a gasoline purchase is computed as the price per gallon for the type of gas purchased, mult

4. "The system should be available 24 hours a day, 7 days a week. 不现实的

10. 对给定的需求示例, 设计功能测试用例

1. 以需求为线索, 开发测试用例套件, 确定输入/输出, 开发测试用例。

输入						预期输出
ID	商品信息	特价	赠品	支付		
TUS1-1	无商品	无	无	无	不做任何处理，关闭销售任务	
TUS1-2	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无			
TUS1-3	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无	85	无找零，系统行为满足后置条件	
TUS1-4	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	商品1特价20		100	找零30，系统行为满足后置条件	
TUS1-5	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	总额特价50以上0.8折		100	找零31，系统行为满足后置条件	
TUS1-6	商品1 (1、1(双)、35) 商品2 (2、1(双)、60)	商品1特价20 总额特价50以上0.8折		100	找零32，系统行为满足后置条件	

2. 制定测试用例

11. 度量 不重要

- 1. 用例的数量
 - 平均每个用例中的场景数量
 - 平均用例行数
 - 在一个尺度下进行分析
- 2. 软件需求数量
- 3. 非功能需求数量
- 4. 功能点数量

11.1. 功能点度量

- 1. 输入数量
- 2. 输出数量
- 3. 查询数量
- 4. 逻辑文件数量
- 5. 对外接口数量

12. 软件设计

- 1. 软件设计是指关于软件对象的设计，是一种设计活动。软件设计既指软件对象实现的规格说明，又指这个规格说明产生的过程。

2. 软件设计活动以需求开发的制品(需求规格说明和分析模型)为基础, 构建软件设计方案描述和原型, 为后期的构造活动提供规划或蓝图。
3. 软件设计兼具工程性和艺术性, 由于软件系统的可变性, 软件设计具有演化性, 也因为软件设计的过程实际上就是一系列决策发生的过程, 软件设计具有决策性。

12.1. 软件设计的核心思想

1. 分解: 横向上将系统分割为几个相对简单的子系统与子系统之间的关系
2. 抽象: 在纵向上聚焦个子系统的接口(这里的接口与实现相对), 可以分离接口和实现, 使得人们更好的关注软件系统本质, 降低复杂度。

12.2. 软件设计的核心层次

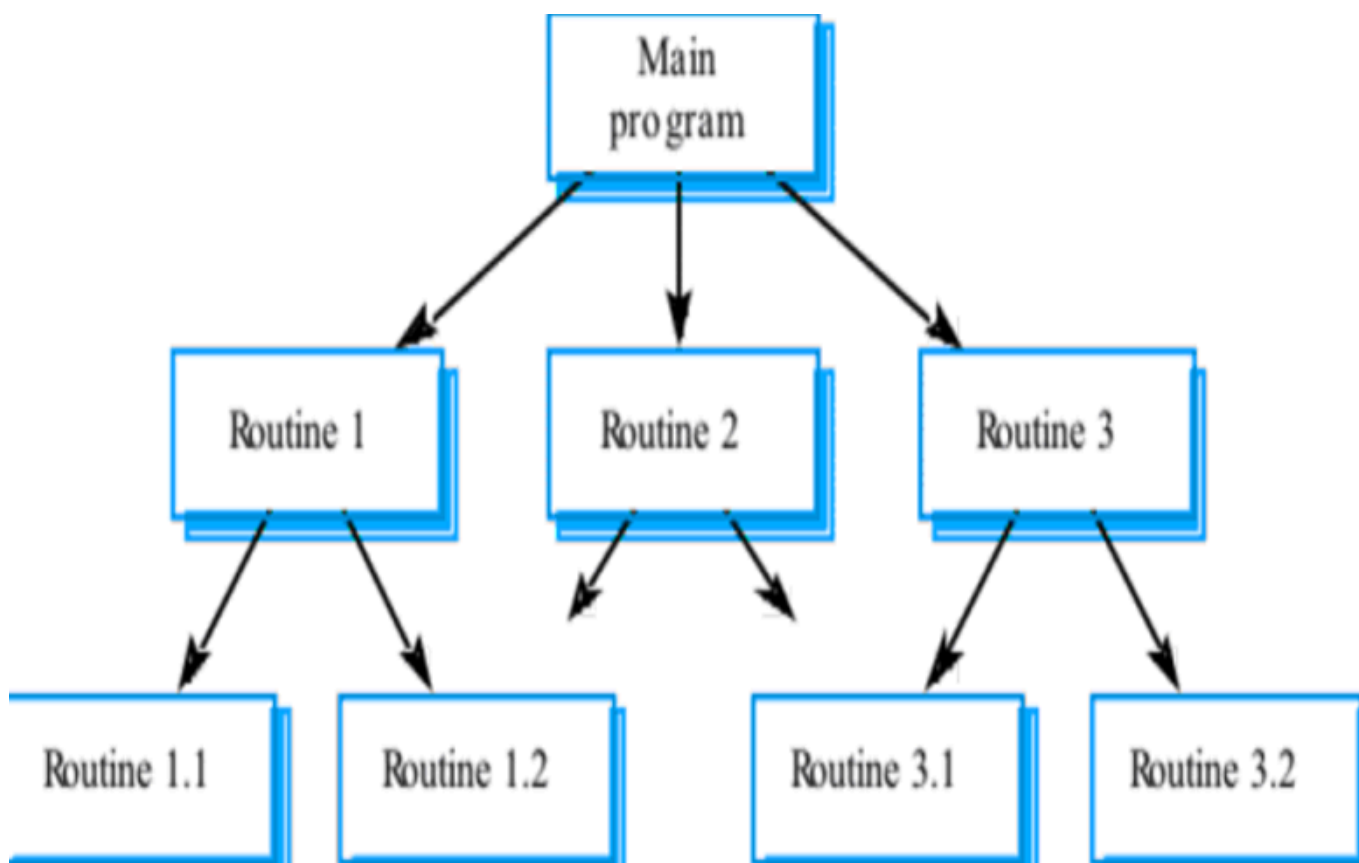
1. 高层设计: 基于反映软件高层抽象的构件设计, 描述系统的高层结构、关注点和设计决策。
 - i. **部件**承载了系统主要的**计算与状态**
 - ii. **连接件**承载部件之间的**交互**
 - iii. 部件与连接件都是抽象的类型定义(就像类定义), 它们的实例(就像类的对象实例)组织构成软件系统的整体结构, **配置**将它们的实例连接起来
2. 中层设计: 更加关注组成构件的模块的设计、导入/导出、过程之间调用关系或者类之间的协作, 模块划分**隐藏**一些程序片段(数据结构+算法)的细节, 暴露接口于外界
3. 低层设计: 深入模块和类的内部, 关注具体的数据结构、算法、类型、语句和控制结构等。

13. 体系结构

13.1. 体系结构概念

1. 软件体系结构是由部件, 连接件, 配置组成的。
 - i. **部件**是软件体系结构的基本组成单位之一, 承载系统的主要功能, 包括处理与数据;
 - a. 原始部件
 - b. 复合部件
 - ii. **连接件**是软件体系结构的另一个基本组成单位, 定义了部件间的交互, 是连接的抽象表示;
 - a. 原始连接件
 - b. 复合连接件: 是由更细粒度的部件和连接件组成。
 - iii. **配置**是对"形式"的发展, 定义了"部件"以及"连接件"之间的关联方式, 将它们组织成系统的总体结构。

13.2. 体系结构风格的优缺点



1. 主程序子程序风格

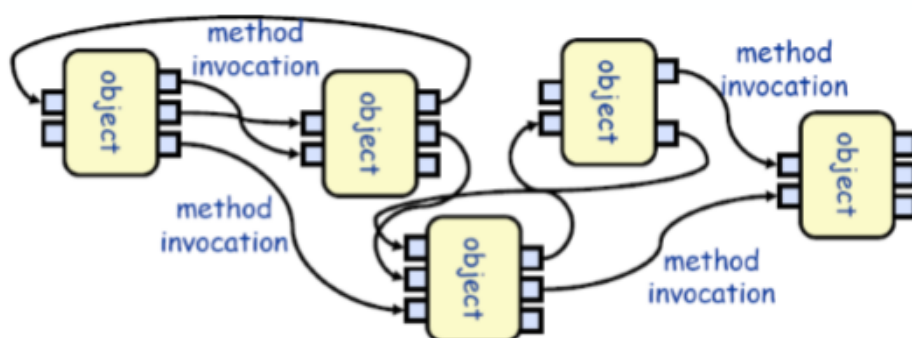
i. 优点:

- a. 流程清晰，易于理解
- b. 强控制性

ii. 缺点:

- a. 程序调用是一种强耦合的连接方式，非常依赖接口
- b. 程序调用的连接方式限制了部件之间的数据交互，可能会导致不必要的公共耦合。

iii. 适用: 可以将系统功能依层次分解为多个顺序执行步骤的系统



2. 面向对象风格

i. 优点:

- a. 内部实现的可修改性(隐藏内部实现)
- b. 易开发、易理解、易复用的结构组织(契合模块化思想)

ii. 缺点:

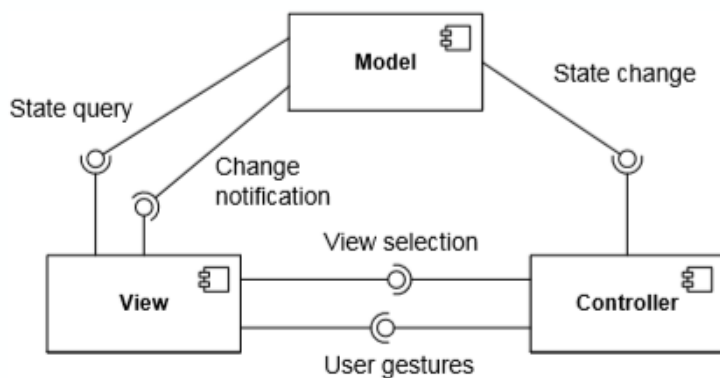
- a. 接口的耦合性(由于方法调用机制，接口的耦合性无法消除)
- b. 标识的耦合性(一个对象要和其他对象交互，必须知道标识符)

- c. 副作用(难以理解、高耦合性以及数据的不一致视图)
- iii. 适用: 适用于那些能够基于数据信息分解和组织的软件系统。



3. 分层风格

- i. 优点:
 - a. 设计机制清晰, 易于理解(抽象层次分离, 隔离复杂度)
 - b. 支持并行开发(层次之间遵守程序稳定的接口)
 - c. 更好的可复用性与内部可修改性(接口的稳定性, 不同层次的部件能够互相替代)
- ii. 缺点:
 - a. 交互协议难以修改(可能需要改变所有的层次, 接口具有强耦合性)
 - b. 性能损失(禁止跨层调用)
 - c. 难以确定层次数量和粒度
- iii. 应用: 适用于主要功能是在不同抽象层次上进行任务分解的复杂处理, 能建立稳定的不同抽象层次之间的稳定交互协议, 没有很高的实时性能要求的系统



4. MVC风格Model-View-Controller:

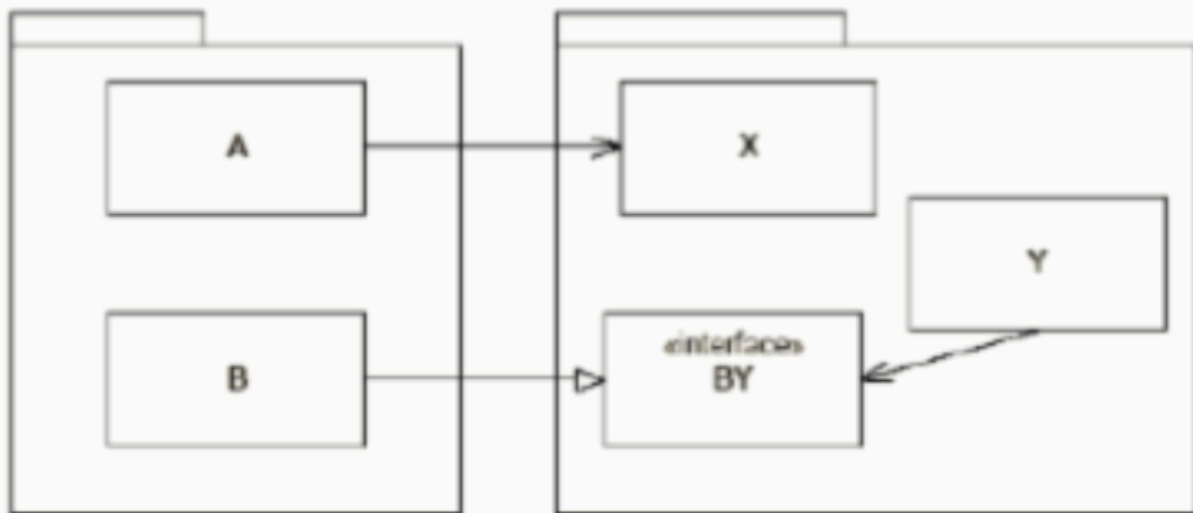
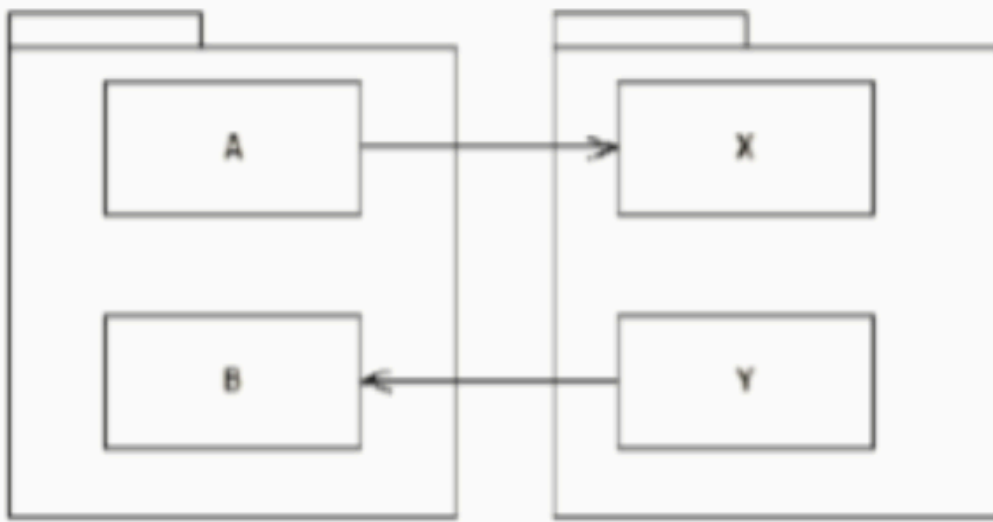
- i. 优点:
 - a. 易开发性:抽象了业务逻辑, 表现和控制机制清晰, 易于开发
 - b. 视图和控制的可修改性
 - c. 适宜于网络系统开发的特征(MVC 不仅允许视图和控制的可修改性,而且其对业务逻辑、表现和控制的分离使得一个模型可以同时建立并保持**多个视图**,这非常适用于网络系统开发)
- ii. 缺点:
 - a. 复杂性: MVC将用户的任务分解成了表现、控制和模型三个部分, 这会增加系统的复杂性, 不利于理解任务实现。
 - b. 模型修改困难, 视图和控制都要依赖于模型

13.3. 软件体系结构设计过程

1. 分析关键需求和项目约束：分析用例文档和需求规格说明书(包含需求规格和项目约束)。注意既要考虑功能性需求，又要考虑非功能性需求，甚至很大意义上体系结构设计是为了满足**非功能性需求**。
2. 通过选择体系结构风格：选择分层风格(信息系统、并行开发、非web应用)，进行评审。
3. 进行软件体系结构逻辑(抽象)设计：产生逻辑包图
4. 依赖逻辑设计进行软件体系结构(实现)设计：产生物理类图
5. 完善体系结构设计：关键类图，持久化数据格式的定义等
6. 添加构件接口：包、重要文件的创建，定义接口
7. 迭代过程3-6

13.4. 包设计原则

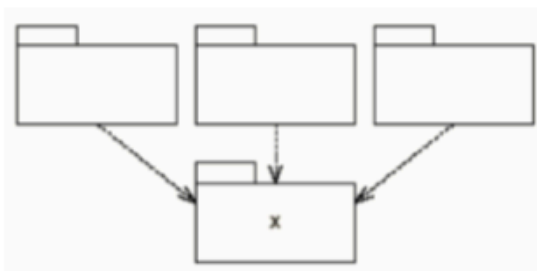
1. 重用发布等价原则(REP):重用的粒度就是发布的粒度
 - i. 为重用器分组组件(类)
 - ii. 单个类通常是不可重用的：几个协作类组成一个包
 - iii. 包中的类应构成可重用和可释放的模块：模块提供一致的功能
 - iv. 减少重新使用者的工作
 - v. 和相关联的类一起发布，而不是单独进行发布
2. 共同封闭原则(CCP):包中所有类对于同一类性质的变化应该是共同封闭的，一个变化若对一个包产生影响，则对该包中的所有类产生影响，而对于其他包不造成任何影响。
 - i. 最小化修改都程序员的影响
 - ii. 包尽可能大，和CRP互斥
 - iii. 方法
 - a. 将具有相似闭包的类分组
 - b. 面向可以预期的变更封闭包
 - c. 将更改限制为几个软件包
 - d. 降低包装释放频率
 - e. 减少程序员的工作量
 - f. 只对可预测的变更有作用，不可预测的变更会为系统带来极大的破坏能力，并且无法进行预测。
3. 共同重用原理(CRP):一个包中的所有类应该是能够共同重用的。
 - i. s根据常见重用对类进行分组：避免给用户不必要的依赖
 - ii. 遵循CRP通常会导致软件包拆分：获得更多，更小，更专注的包
 - iii. 减少重新使用者的工作
 - iv. 包尽可能小，和CCP互斥
4. 无环依赖原则(ADP):在包的依赖关系图中不能存在环。必须是有向无环图。
 - i. 第一种单环，DIP依赖倒置即可解决
 - ii. 第二种互环，A依赖B，且B依赖A



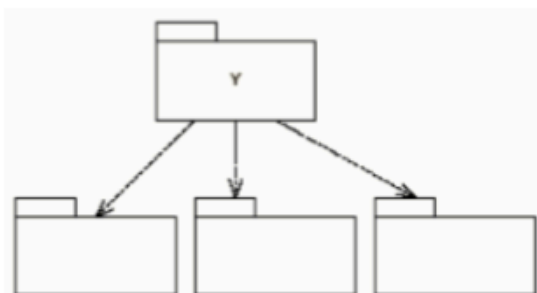
5. 稳定依赖原则(SDP):朝着稳定(别人的修改不影响我)的方向进行依赖

- i. C_a :输入耦合度, 包外部依赖本包的类个数
- ii. C_e :输出耦合度, 包内部依赖于包外部的类的个数
- iii. 不稳定性: $I = \frac{C_e}{C_e + C_a}$, 越小越稳定

● X is stable



● Y is instable



6. 稳定抽象原则(SAP):包的抽象程度应该和其稳定程度一致

- i. 稳定的包应该是抽象的包
- ii. 不稳定的包应该是具体的包
- iii. N_a :包中抽象类个数
- iv. N_c :包中所有类个数
- v. 抽象度 $A = \frac{N_a}{N_c}$

7. 前三条描述的是依赖性，后三条描述的是耦合性

8. 包设计过程:

- i. 开发包(构件)设计
- ii. 运行时的进程
- iii. 物理部署

13.5. 了解的图片

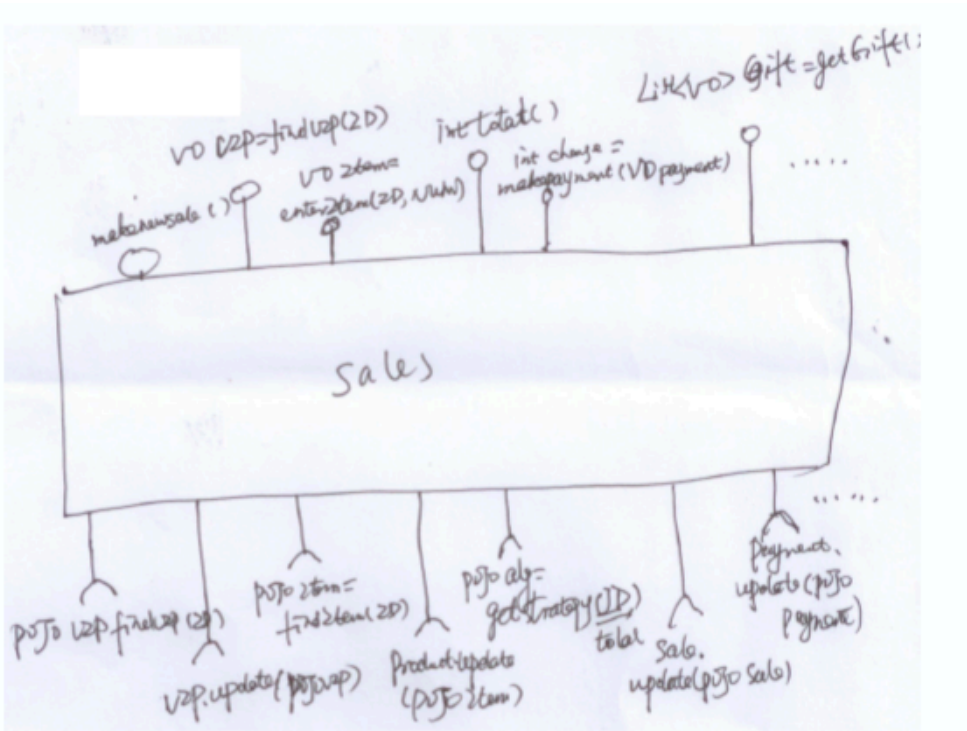
- 1. 进程图
- 2. 部署图

13.6. POJO和VO

- 1. POJO: Java实体
- 2. VO: 值对象, 逻辑层向展示层传递

13.7. 体系中构建的之间的接口的定义 重要

- 1. 首先确定模块对外接口
- 2. 然后确定接口的规范



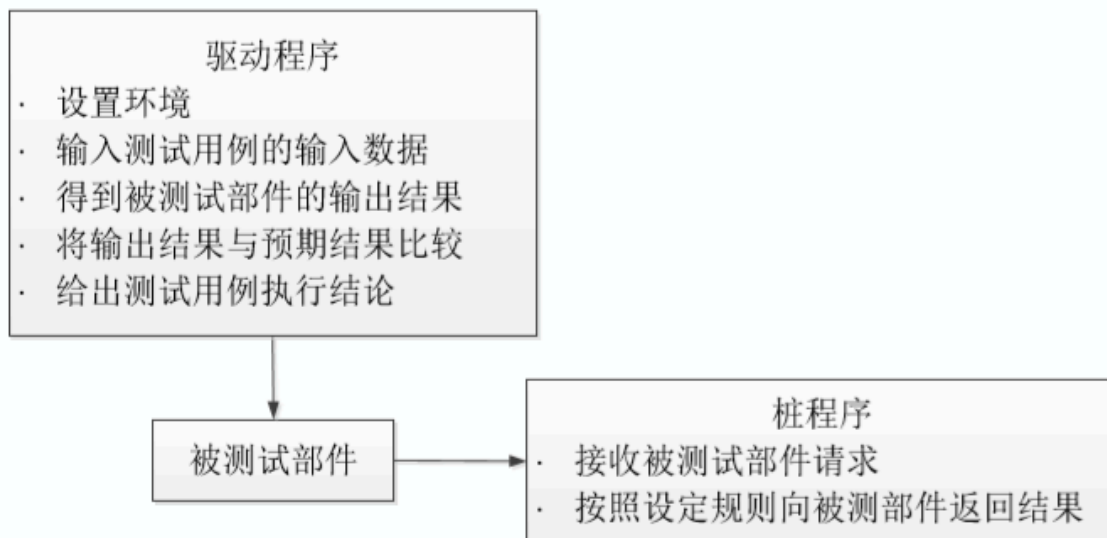
提供的服务（供接口）		
LoginController.login	语法	public ResultMessage login(long id, String password);
	前置条件	用户输入有效的用户编号和密码。user 成员变量初始化为 null。
	后置条件	根据 id 查找是否存在相应的用户，根据用户信息验证其密码正确性，如果通过验证记录当前用户。
需要的服务（需接口）		
服务名	服务	
UserDataService.find	根据 ID 进行查找用户信息	

13.8. 集成的策略 不重要

- 1. 大爆炸式集成
- 2. 增量式
 - i. 自顶向下式
 - ii. 自底向上式
 - iii. 三明治式
 - iv. 持续集成

13.9. 体系结构开发集成测试用例

- 1. Stub：为了完成程序的编译和连接而使用的暂时代码
- 2. Driver：驱动一个测试,就是驱动一个Service的行为



编写驱动程序，在桩程序帮助下进行集成测试

3. 例子

Stub定义

```
public interface SalesBLService_Stub implements SalesBLService {
    String commodityName;
    double commodityPrice;
    double commodityDiscount;
    CommodityPromotion promotion;
    String memberName;
    int memberPoint;
    double total;
    double change;

    public SalesBLService_Stub(String cn, double cp, double cd, CommodityPromotion p, String
mn, int mp, double t, double c){
        commodityName = cn;
        commodityPrice = cp;
        commodityDiscount = cd;
        promotion = p;
        memberName = mn;
        memberPoint = mp;
        total = t;
        change = c;
    }
    //销售界面得到商品和商品促销的信息
    public CommodityVO getCommodityById(int id){
        return new CommodityVO(name, price, commodityDiscount);
    }
    public ArrayList<CommodityPromotionVO> getCommodityPromotionListById(int
commodityID){
        ArrayList<CommodityPromotionVO> commodityPromotionList = new
ArrayList<CommodityPromotionVO>();
        commodityPromotionList.add(new CommodityPromotionVO(promotion));
        return commodityPromotionList;
    }
}
```

Stub使用

```
public class SalesView{
    .....
    SalesBLService salesBl = new SalesBLService_Stub ("Cup",10.0,2.0, new
CommodityPromotion(), "Karen", 500, 57.0, 3.0);
    .....
}
```


Driver定义

```
public class SalesBLService _Driver{
    public void drive(SalesBLService salesBLService){
        ResultMessage result = salesBLService.addMember(0911024);
        If(result== ResultMessage.Exist) System.out.println("Member exists\n");
        .....
        salesBLService.endSales();
    }
}
```

Driver使用

```
public class Client{
    public static void main(String[] args){
        SalesBLService salesController = new SalesController();
        SalesBLService _Driver driver = new SalesBLService _Driver (salesController);
        driver.drive(salesController);
    }
}
```

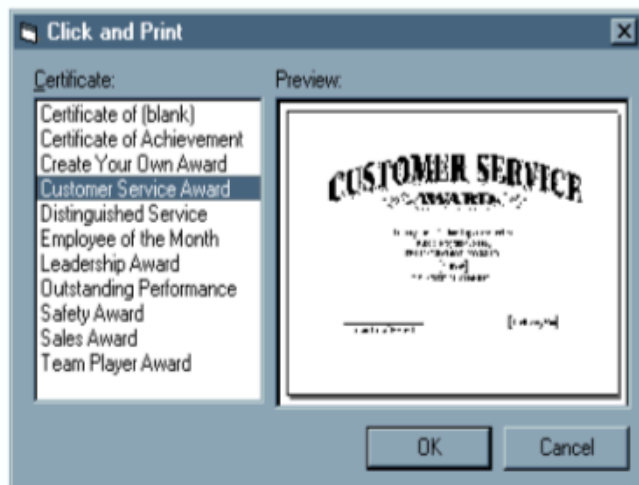
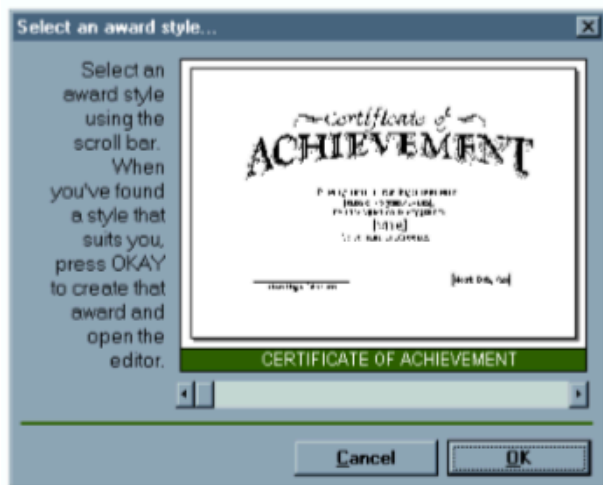
14. 人机交互设计

14.1. 可用性

1. 可用性不是用户界面的单一属性，它包含以下维度。
 - i. 易学性：新手用户容易学习，能够很快使用系统。
 - ii. 效率：熟练的用户可以高效使用它
 - iii. 易记性：使用过软件系统的用户，能够有效记忆或快速重新学会使用该系统。(超市可以缓存之前的信息)
 - iv. 出错率：几乎没有错误，可以从错误中快速恢复
 - v. 主观满意度：让用户有良好的体验。

14.2. 人机交互设计原则

1. 简洁设计(7±2原则)：文字不如图形
2. 一致性设计(实际模型和人的精神模型一致)，如下是一个不一致的例子(cancel和ok的位置问题)



3. 低出错率设计(包括不适当的菜单功能灰色屏蔽、检查输入，出现错误的时候帮助用户修正错误，而不是重新填写，每一个输入都尽快的显示问题)
4. 易记性设计
 - i. 减少短期记忆的负担
 - ii. 使用逐层递进的方式展示信息
 - iii. 使用直观的快捷方式
 - iv. 设置有意义的默认值
5. 可视化设计
 - i. 按照任务模型设计界面隐喻，同时不要把软件系统的内部构造机制暴露给用户
 - ii. 可视化设计还应该基于界面隐喻，尽可能地把功能和任务细节表现出来。

14.3. 精神模型

1. **精神模型**就是用户进行人机交互时头脑中的任务模型。依据精神模型可以进行**隐喻**(Metaphor)设计：
 - i. 隐喻又被称为视觉隐喻，是视觉上的图像，但会被用户映射为业务事物。用户在识别图像时，会依据隐喻将控件功能与已知的熟悉事物联系起来，形成任务模型；
 - ii. 隐喻本质上是在用户已有知识的基础上建立一组新的知识，实现界面视觉提示和系统功能之间的知觉联系。

14.4. 差异性

1. 新手用户
 - i. 是对业务不熟悉的人
 - ii. 例如新员工或者新接触系统的人。为新手用户设计系统时要关注易学性，进行业务导航，尽量避免出错。如果一个系统的大多数用户都是新手用户，整个系统的人机交互设计都要侧重**易学性**。
2. 专家用户
 - i. 是能够熟练操作计算机完成业务任务的人，一般都是长时间使用软件系统并且计算机操作技能熟练的人。
 - ii. 为专家用户设计系统时，要关注效率。如果一个系统的大多数用户都是专家用户，整个系统的人机交互设计都要侧重**效率**。
3. 熟练用户：是介于新手用户和专家用户之间的人。为熟练用户设计人机交互系统要在易学性和效率之间进行折中。
4. 好的人机交互应该为不同的用户群体提供差异化的交互机制。
 - i. 既为新手用户提供易学性高的人机交互机制(图形界面)
 - ii. 又为专家用户提供效率高的人机交互机制(命令行、快捷方式、热键)

14.5. 导航

- 1. 导航是为了给用户提供一个很好的完成任务的入口，好的导航保证这个入口很符合人的精神模型。
- 2. 全局导航控件包括窗口、菜单、列表、快捷方式、热键等等。
- 3. 局部导航通过安排布局细节，制造视觉上的线索来提供导航。

14.6. 反馈

- 1. 反馈能够避免用户进行错误的操作，让用户明确自己的等待过程的长度。
- 2. 时间情况
 - i. 打字、光标移动、鼠标定位：50 ~ 150毫秒
 - ii. 简单频繁的任务：1秒
 - iii. 普通的任务：2 ~ 4秒
 - iv. 复杂的任务：8 ~ 12秒

14.7. 协作式设计

- 1. 人和计算机是人机交互的方法，其中人的因素是比较固定的，一定时期内不会发生大的变化，所以要让两者交互顺畅，就需要让计算机更多地适应人地因素，这也是人机交互设计以用户为中心的根本原因
- 2. 这种调整计算机因素以更好地适应并帮助用户的设计方式被称为协作式设计。

14.8. 小问题

- 1. 不要暴露内部结构:ZIP

名称	特征	人机交互控件	适合场景
批处理	所有的命令预先设置好后，由机器直接一次性执行完毕，中途用户没法进行任何干涉。	不接受输入 无交互 行文本显示（一维输出）	不需要人机交互
命令行	命令模式是一维的交互方式。用户输入一条命令，机器执行一次。	键盘输入（命令） 命令行交互 行文本显示（一维输出）	执行固定任务的熟练用户。
全屏	全屏模式下，通过表单、菜单、导航键和用户进行交互	键盘输入（命令、热键、信息） 菜单、表单交互 可视化输出（菜单、表单，二维全屏输出）	有限任务下熟练用户
图形化	用户利用窗口、菜单、按钮、定位设备（如鼠标等）和机器交互。而且交互操作的方式也是面向对象的，多是可以直观感受到的实物的隐喻。	键盘、鼠标输入 直接操纵（所见即所得）交互 可视化输出（各类可视化控件，多窗口重叠产生二维半输出）	操作技能不熟练的用户
多维交互	正在发展和逐渐出现的交互方式。其中可能会用到声音、视频等机制。更易用和更个性化。	多维输入 多维输出	

- 2. 菜单系统
 - i. 下拉式菜单：需要空间，可预测
 - ii. 弹出式菜单：灵活，定制，让用户惊讶

iii. 减少用户需要知道的命令，打字少，避免用户错误，提供上下文参考。

3. 黄金法则

- i. 让用户掌控一切
- ii. 减少用户的内存负载
- iii. 始终如一

15. 详细设计

15.1. 详细设计的出发点

1. 软件详细设计是在软件体系结构设计之后进行，以需求开发的结果(需求规格说明和需求分析模型)和软件体系结构的结果(软件体系结构设计方案与原型)为出发点。

15.2. 职责

1. 职责是执行任务(操作职责)或维护某些数据(数据职责)的义务。
 - i. 行为职责通常由行为来履行。
 - ii. 数据职责通常由属性来完成。
 - iii. 可能会涉及到类之间的协作。
2. 职责分类
 - i. 属性职责:对象的状态
 - ii. 行为职责:对象的行为
3. 通过职责建立静态设计模型

15.3. 职责分配

1. 将比较大的职责给很多对象。

15.3.1. GRASP原则(一般职责分配软件模式)

1. 特点:
 - i. 低耦合：分配一个职责要保证低耦合度
 - ii. 高耦合：分配一个职责的时候要保持类的高聚合度
 - iii. 信息专家：将一个职责分配给专家-履行职责所必须的信息的类
 - iv. 创建者：创建规则在后面
 - v. 控制者：控制规则在后面(避免大多数信息由一个类发出、组件相对较小、行为职责和数据绑定、职责单一)
2. 拇指原则:研究出来良好内聚性、耦合性和稳定性的替代方案。
3. 信息专家:统一提供对外的接口，而不是某一个具体的接口，数据和行为不要分开
4. 创建规则:如果有以下情况，则由创建者分配B类创建A类实例的职责：
 - i. B 聚集了 A 对象
 - ii. B 包含了 A 对象
 - iii. B 记录了 A 的实例
 - iv. B 要经常使用 A 对象
 - v. 当 A 的实例被创建，B具有传递给A的初始化数据(也就是 B 是创建 A 的实例这项任务的信息专家)

vi. 在有选择的地方，更喜欢B聚合或包含A对象

表 12-4 对象创建者

优 先	场 景	创建地点	创建时机	备 注
高 ↑ ↓ 低	唯一属于某个整体的密不可分的一部分（组合关系）	整体对象的属性定义和构造方法	整体对象的创建	例如，销售的业务逻辑对象由销售页面对象创建
	被某一对象记录和管理（单向被关联）	关联对象的方法	业务方法的执行中对象的生命周期起始点	例如，连接池管理对象需要负责创建连接池对象
	创建所需的数据被某个对象所持有	持有数据的对象的业务方法	业务方法的执行中	也可以考虑在此持有数据对象不了解创建时机时，由别的对象创建，由它来初始化
	被某个整体包含（聚合关系）	整体对象的业务方法（非构造方法）	业务方法的执行中	如果某个对象有多个关联，优先选择聚合关联的整体对象。如果某个对象有多个聚合关联的整体对象，则考查整体对象的高内聚和低耦合来决定由谁创建
	其他			通过高内聚和低耦合来决定由谁创建

- 第一个(组合关系)
- 第二个(单向被关联):比如访问数据库，你要访问的时候，我就给一个访问对象来使用，不用的时候归还就行。
- 第三个(持有必要数据):根据业务的情况决定什么时候被创建，有时候B可以创建但是不知道什么时机来创建，如果C知道，那么我们可能让C创建对象，然后B进行初始化
- 第四个(聚合关系):关系比较多，要看时机等什么时候合适

15.4. 协作

1. 程序中的对象互相协作，完成一个较大的职责。
2. 通过协作建立动态设计模型

15.4.1. 协作的测试:Mock Object

```

public class TotalIntegrationTester {
    @Test
    public void testTotal () {
        MockCommodity commodity1 = new MockCommodity (50);
        MockCommodity commodity2 = new MockCommodity (40);
        SalesLineItem salesLineItem1 =
            new SalesLineItem (commodity1, 2);
        SalesLineItem salesLineItem2
            = new SalesLineItem (commodity2, 3);

        Sales sale=new Sales();
        sale.addSalesLineItem(salesLineItem1);
        sale.addSalesLineItem(salesLineItem2);

        assertEquals (220, sale.total () );
    }
}

public class MockCommodity extends Commodity{
    double price;
    public MockCommodity ( double p){
        price=p;
    }
    public getPrice () {
        return price;
    }
}

```

1. Mock Object类似于Stub，但是更加简单

15.4.2. 集成测试

```

public class TotalIntegrationTester {
    @Test
    public void testTotal () {
        MockCommodity commodity1 = new MockCommodity (50);
        MockCommodity commodity2 = new MockCommodity (40);
        SalesLineItem salesLineItem1 =
            new SalesLineItem (commodity1, 2);
        SalesLineItem salesLineItem2
            = new SalesLineItem (commodity2, 3);

        Sales sale=new Sales();
        sale.addSalesLineItem(salesLineItem1);
        sale.addSalesLineItem(salesLineItem2);

        assertEquals (220, sale.total () );
    }
}

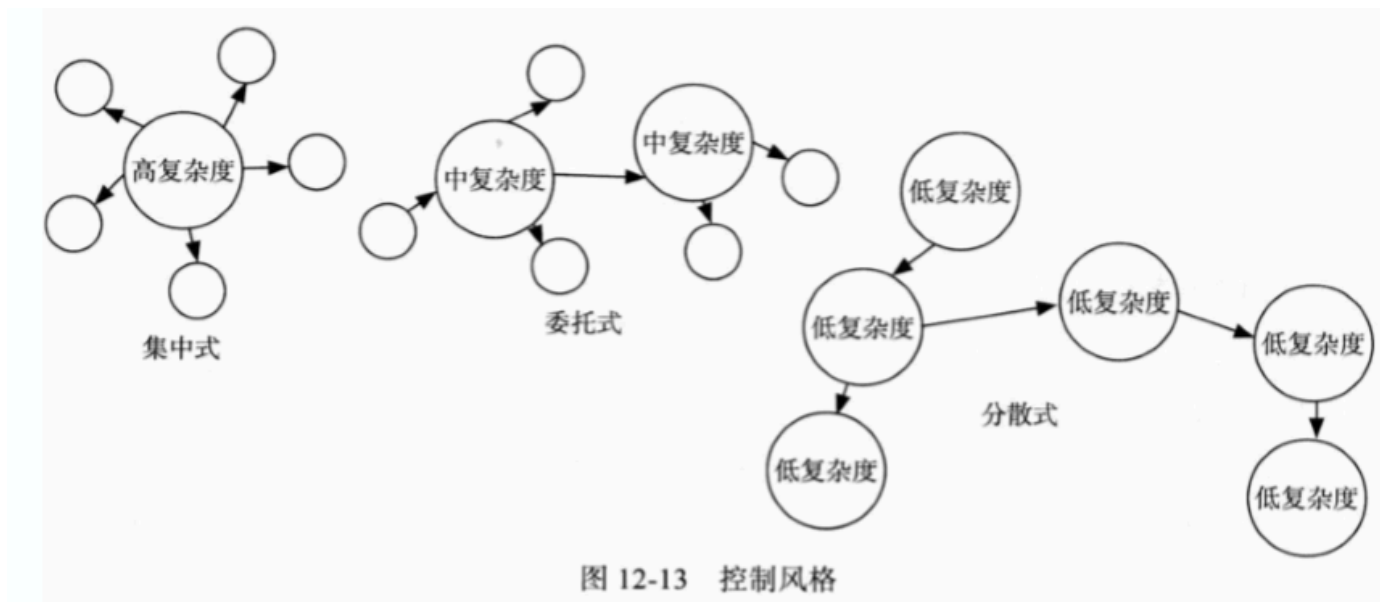
```

集成测试代码

15.5. 控制风格

1. 集中式控制风格：一个额外的控制器记录系统行为的所有逻辑，容易找到做出决定的位置，但是控制器比较庞大，复杂且难以理解，都是控制器调用他者
2. 委托式控制风格：做出决策的对象不知有一个，职责的分解决定了控制对象的层次。

3. 分散式控制风格：系统行为的逻辑通过对象网络**广泛传播**，隐藏信息很难，内聚性比较差

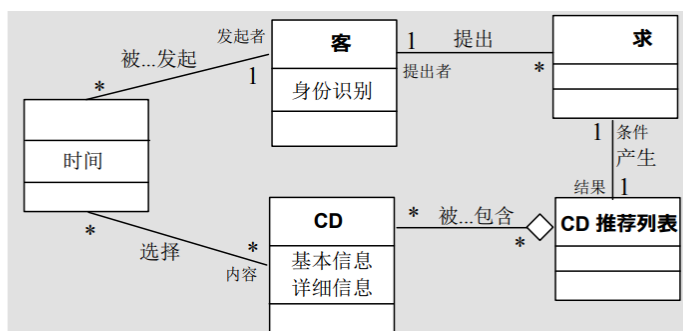
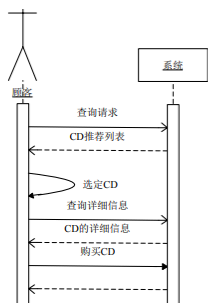


15.6. 部分题目

1. 给定分析类图(概念类图)、系统顺序图和设计因素描述，建立设计类图(设计类图就是满足这三种控制风格的类图)/系统顺序图

用例描述:

1. 顾客向系统提起查询请求
2. 系统根据请求为顾客提供一个CD的推荐列表
3. 顾客在推荐列表选定一个CD，然后要求查看更详细的信息
4. 系统为顾客提供选定CD的详细信息
5. 顾客购买选定CD
6. 顾客离开。



画出设计类图，顺序图（集中式控制的方式）

16. 内聚和耦合

1. 内聚：表达的是一个模块内部的联系的紧密性：包括信息内聚、功能内聚、通信内聚、过程内聚、时间内聚、逻辑内聚和偶然内聚。
2. 耦合：描述的是两个模块之间关系的复杂程度：包括内容耦合，公共耦合，重复耦合，控制耦合，印记耦合，数据耦合

类 型	内 聚 性	解 释	例 子
偶然内聚	最低 ↑ ↓ 最高	模块执行多个完全不相关的操作	把下列方法放在一个模块中：修车、烤面包、遛狗、看电影
逻辑内聚		模块执行一系列相关操作，每个操作的调用由其他模块来决定	把下列方法放在一个模块中：开车去、坐火车去、坐飞机去
时间内聚		模块执行一系列与时间有关的操作	把下列方法放在一个模块中：起床、刷牙、洗脸、吃早餐
过程内聚		模块执行一些与步骤顺序有关的操作	把下列方法放在一个模块中：守门员传球给后卫、后卫传球给中场球员、中场球员传球给前锋、前锋射门
通信内聚		模块执行一系列与步骤有关的操作，并且这些操作在相同的数据上进行	把下列方法放在一个模块中：查书的名字、查书的作者、查书的出版商
功能内聚		模块只执行一个操作或达到一个单一目的	下列内容都作为独立模块：计算平方根、决定最短路径、压缩数据
信息内聚		模块进行许多操作，各个都有各自的入口点，每个操作的代码相对独立，而且所有操作都在相同的数据结构上完成	比如数据结构中的栈，它包含相应的数据和操作。所有的操作都是针对相同的数据结构

内聚

类 型	耦 合 性	解 释	例 子
内容耦合	最高 ↑ ↓ 最低	一个模块直接修改或者依赖于另一个模块的内容	程序跳转 GOTO ；某些语言机制支持直接更改另一个模块的代码；改变另一个模块的内部数据
公共耦合		模块之间共享全局的数据	全局变量
重复耦合		模块之间有同样逻辑的重复代码	逻辑代码被复制到两个地方
控制耦合		一个模块给另一个模块传递控制信息	传递“显示星期天”。传递模块和接收模块必须共享一个共同的内部结构和逻辑
印记耦合		共享一个数据结构，但是却只用了其中一部分	传递了整个记录给另一个模块，另一个模块却只需要一个字段
数据耦合		两个模块的所有参数是同类型的数据项	传递一个整数给一个计算平方根的函数

耦合

16.1. 结构化内聚

1. 偶然内聚:完全不相关
2. 逻辑内聚:执行一系列操作，具体操作交给具体模块(如switch)，解决控制耦合的方法

```
public void sample( String flag ) {  
    switch ( flag ){  
        case ON:  
            .....  
            break;  
        case OFF:  
            .....  
            break;  
        case CLOSE:  
            .....  
            break;  
    }  
}
```

3. 时间内聚:执行一系列和时间有关的操作，比如构造方法和析构方法

```
Public class foo {  
    Private string name;  
    Private int size;  
    //constructor  
    public void foo(){  
        this.name = "Not Set";  
        this.size = 12;  
    }  
    //destructor  
    Public void ~foo() {  
        delete[] name;  
        delete size;  
    }  
}
```

4. 过程内聚:执行一系列与步骤顺序有关的操作

```

void MonthEnd()
{
    Report ExR = InitExpenseReport();
    Report rr = InitRevenueReport();
    Report EmpR = InitEmployeeReport();

    EmpR.Init();
    rr.Init();
    ExR.SetEmployees(true);

    if (ExR.GetReportParams())
        EmpR.GetReportParams();

    SendToPrinter(rr);
    SendToPrinter(ExR);
    SendToPrinter(EmpR);
}

```

5. 通信内聚:执行一系列与步骤顺序有关的操作, 并且这些操作是在相同的数据结构上。

```

Public class Calculate {
    Public int product;
    public void product(int a, int b){
        product = a*b;
        ....
        save(product);
        ....
    }
    Public void save(int product) {
        \\ code to store value into database
        ...
    }
}

```

6. 功能内聚:模块只执行一个操作或达到一个单一目的

```
public int commission(int sale, long percentage){  
    int com;  
    //calculate commission  
    return com;  
}
```

7. 信息内聚:模块进行许多操作, 各自有各自的入口, 相互独立, 且在相同的数据结构上完成操作。

```
Public interface Addressee{  
    .....  
    Public abstract String getName ();  
    Public abstract String getAddress ();  
    ....  
}
```

```
Public class Employee implements Addressee {....}
```

8. 偶然内聚和逻辑内聚不可以接受

16.2. 结构化耦合

1. 数据耦合:两个模块的所有参数是同类型的数据项(发送正好)

```
Public class Receiver {  
  
    public void message( MyType X ){  
  
        ...  
        X.doSomethingForMe( Object data );  
        ...  
    }  
}
```

2. 印记耦合:两个模块共享一个数据结构, 但是只用了其中一个部分(发送多了)

Code Snippet 1

```
void validate_checkout_request(input_form i)  
{  
    if (!valid_string(i.name)) {  
        error_message("Invalid name");  
    }  
  
    if (!valid_string(i.book)) {  
        error_message("Invalid book name");  
    }  
  
    if (!valid_month(i.date)) {  
        error_message("Invalid month");  
    }  
}  
  
int valid_month(date d)  
{  
    return d.month >= 1 && d.month <= 12;  
}
```

Code Snippet 2

```

void validate_checkout_request(input_form i)
{
    if (!valid_string(i)) {
        error_message("Invalid name");
    }

    if (!valid_string(i)) {
        error_message("Invalid book name");
    }

    if (!valid_month(i)) {
        error_message("Invalid month");
    }
}

int valid_month(input_form i)
{
    return i.date.month >= 1 && i.date.month <= 12;
}

```

```

public class Employee{
    public String name, emailID.

    ...
}

```

```

public class Mailer{
    public void sendEmail(Employee e, String text)
    {...}

    ...
}

```

3. 控制耦合:一个模块给另一个模块传递了控制信息(除了数据还有控制信号, 并且违反了单一职责原则)
- i. 主要是case中不应该出现具体实现的代码, 应该抽象成方法

Code Snippet 3

```
void validate_checkout_request(input_form i)
{
    if (!valid(i.name, STRING)) {
        error_message("Invalid name");
    }

    if (!valid(i.book, STRING)) {
        error_message("Invalid book name");
    }

    if (!valid(i.date, DATE)) {
        error_message("Invalid month");
    }
}

int valid(string s, int type)
{
    switch (type) {
        case STRING:
            return strlen(s) < MAX_STRING_SIZE;
        case DATE:
            date d = parse_date(s);
            return d.month >= 1 && d.month <= 12;
    };
}
```

```
public routineX(String command) {
    if (command.equals("drawCircle")){
        drawCircle();
    }
    else {
        drawRectangle();
    }
}
```

4. 重复耦合:一个模块有逻辑上相同的重复代码

Code Snippet 6 -- 重复耦合

```
void
validate_checkout_request(input
_form i)
```

```
{
    int len = 0;

    boolean valid_string = false;
```

```
    len = i.name.length();
```

```
    char arr1[] = new
char[len];
```

```
    for(char c: arr1){
```

```
        if c是小写字母
        valid_string = true;
```

```
    }
    if (!valid_string) {
        error_message("Invalid
name");
    }
```

```
    len = i.book.length();
```

```
    char arr2[] = new
char[len];
```

```
    for(char c: arr2){
```

```
        if c是小写字母
        valid_string = true;
```

```
    }
```

```
    if (!valid_string) {
```

```
        error_message("Invalid
book name");
```

```
    }
```

```
    if (!valid_month(i.date)) {
```

```
        error_message("Invalid
month");
```

```
    }
```

```
}
```

5. 公共耦合:模块之间共享全局数据结构

Code Snippet 4

```
string patron_name, book_name;
date checkout_date;
```

```
void validate_checkout_request(input_form i)
```

```
{
    patron_name = i.name;
    if (!valid_string()) {
        error_message("Invalid name");
    }

    book_name = i.book;
    if (!valid_string()) {
        error_message("Invalid book name");
    }

    checkout_date = i.date;
    if (!valid_month()) {
        error_message("Invalid month");
    }
}
```

```
int valid_month()
```

```
{
    return checkout_date.month >= 1 && checkout_date.month <= 12;
}
```

```
int x;

public class myValue{

    public void addValue(int a){
        x = x+ a;
    }
    public void subtractValue(int a){
        x = x- a;
    }

}
```

6. 内容耦合:一个模块直接修改另一个模块的内容(成员变量、函数等)

7. 控制耦合及以上可以接受

- 隐式耦合(面向对象耦合)

Code Snippet 5

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i.name)) {
        i.string = "Invalid name";
        error_message();
    }

    if (!valid_string(i.book)) {
        i.book = "Invalid book name";
        error_message();
    }

    valid_month(i.date);
}

void valid_month(date d)
{
    if (d.month < 1) {
        d.month = 1;
    }

    if (d.month > 12) {
        d.month = 12;
    }

    return 1;
}
```

```
public class Vector3D{
    public int x, y, z;
    ...
}
public class Arch {
    private Vector3D baseline;
    ...
    void slant(int newY){
        baseline.x = 10;
        baseline.y = 13;
    }
}
```

16.3. 面向对象耦合：

- 1. 隐式耦合：不设置局部变量，级联调用
- 2. 显示耦合：可以通过委托方式完成
- 3. 继承耦合

表 14-2 继承耦合

类 型		耦 合 性	解 释
修改（modification）	规格	最高 ↑ ↓ 最低	子类任意修改从父类继承回来的方法的接口
	实现		子类任意修改从父类继承回来的方法的实现
精化（refinement）	规格		子类只根据已经定义好的规则（语义）来修改父类的方法，且至少有一个方法的接口被改动
	实现		子类只根据已经定义好的规则（语义）来修改父类的方法，但只改动了方法的实现
扩展（extension）			子类只是增加新的方法和成员变量，不对从父类继承回来的任何成员进行更改
无（nil）			两个类之间没有继承关系

16.4. 这部分区分需要解释

17. KWIC的实现

- 1. MS:主程序子程序，分别调用
- 2. OO:职责和协作
- 3. Pipe Filter:数据进入过滤器，通过管理连接过滤器，每一个过滤器是独自运行的，多线程
- 4. Event:基于事件，订阅者和分发者

18. 信息隐藏

每一个模块都隐藏了这个模块中关于重要设计决策的实现，以至于只有这个模块的每一个组成部分才能知道具体的细节

18.1. 基本思想

每个模块都隐藏了重要设计决策的实现，因此只有该模块的组成部分才知道详细信息：特别是如果存在所有可能的设计更改的列表-隐藏假设列表

18.2. 秘密

1. 主要秘密：主要秘密描述的是这个模块所要实现的**用户需求**。是设计者对用户需求的实现的一次职责分配。有了这个描述以后，我们可以利用它检查我们是否完成所有的用户需求，还可以利用它和需求优先级来决定开发的次序。
2. 次要秘密：次要秘密描述的是这个模块在实现职责时候所涉及的具体的**实现细节**。包括数据结构，算法，硬件平台等信息。

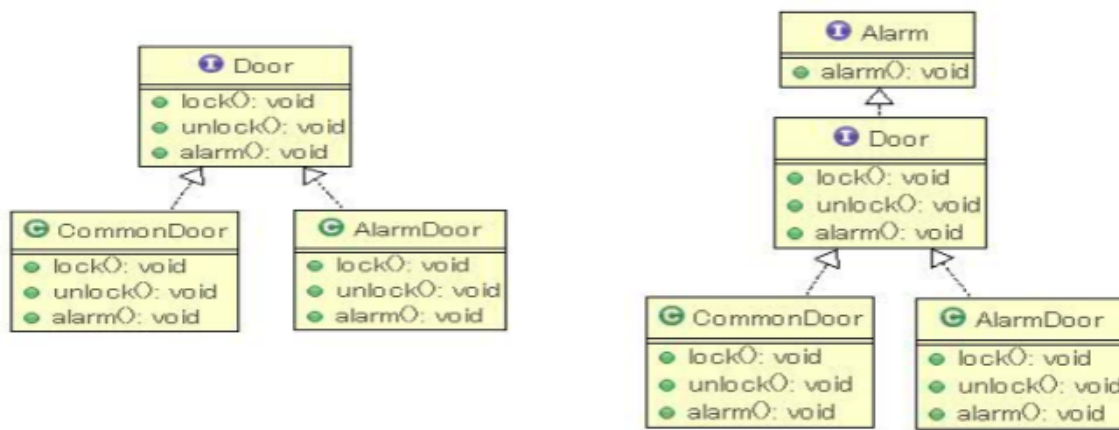
18.3. 两种常见的信息隐藏

1. 根据需求分配的职责，因为实践表明，需求是经常变化的，频率和幅度都很大；
2. 内部实现机制，常见的变化主题包括硬件依赖，输入输出形式，非标准语言特征和库，负责的设计和实现，复杂的数据结构，复杂的逻辑，全局变量。数据大小限制等。

19. 设计原则汇总

1. 原则一:全局变量是被认为是有害的
 - i. 公共耦合是有风险，而且会增加潜在连接数量
2. 原则二:如果没有特殊要求，让代码清晰一点:
 - i. 让代码兼顾明确和可修改性
3. 原则三:避免重复
 - i. 面向接口编程，而不是重复地写逻辑上一致的代码
4. 原则四:面向接口编程
5. 原则五:迪米特法则
 - i. 你可以自己玩。(this)
 - ii. 你可以玩自己的玩具，但不能拆开它们(自己的成员变量)
 - iii. 你可以玩送给你的玩具。(方法)
 - iv. 你可以玩自己制作的玩具。(自己创建的对象)
 - v. 强调的是不能出现a.b.Methods这类情况，不能人去动狗的腿，而是人命令狗，狗动腿
6. 原则六:接口隔离原则(ISP)/也叫接口最小化原则
 - i. 将大的接口拆解成几个小的接口。
 - ii. 这样可以有效避免出现不必要的依赖。
7. 原则七:里氏替换原则(LSP)

- i. 所有派生类都必须可以替代其基类
- ii. 派生类的前置条件更弱，后置条件更强
- iii. 解决方案:在父类中添加方法或者函数，或者拆分接口



- 应该分为Door和Alarm两个接口，第一个CommonDoor不应该实现alarm

8. 原则八:组合代替继承

- i. MyStack 组合 Vector

9. 原则九:单一职责原则:一个类只能有一个改变的理由

10. 原则十:最小化类和成员的可访问性(x是可见)

- 是不是需要public
- 类声明前没有public，则包内可见
- 方法生命前没有public，则包内可见
- public修饰是全局可见

Access Specifier	Class	Package	Subclass	World
private	x			
None	x	x	x*	
protected	x	x	x**	
public	x	x	x	x

类和成员的可访问性

* Subclasses within the same package can also access members that lack access specifiers (default or package-private visibility). An additional requirement for access is that the subclasses must be loaded by the class loader that loaded the class containing the package-private members. Subclasses in a different package cannot access such package-private members.

** To reference a protected member, the accessing code must be contained either in the class that defines the protected member or in a subclass of that defining class. Subclass access is permitted without regard to the package location of the subclass.

11. 原则十一:开放/封闭原则(OCP)

- i. 对扩展开放：模块的行为可以被扩展，比如新添加一个子类
- ii. 对修改关闭：模块中的源代码不应该被修改
- iii. RTTI(运行时类型信息违反了开闭原则LSP)，就是如果都有的话则抽象成基类的方法。

12. 原则十二:依赖倒置原则(DIP):高级模块不应依赖于低级模块：两者都应依赖抽象。



分析下图请说明：（1）它正确运用了哪条面向对象设计原则，请详细解释这条原则（5分）；（2）User类的内部是哪种层次的内聚？请详细描述这个层次内聚的特征（5分）；

19.1. 内聚耦合度量(了解)

1. 对象类之间的耦合：CBO = 该类访问其他类的成员方法的数量 + 其他类的成员访问该类的成员方法的数量
2. 数据抽象耦合：DAC = 统计一类包含的其他类的其他类的实例的数量，不包括继承关系带来的实例引用
3. 输入耦合度 C_a : 在此类之外依赖于这类内部的类的数量
4. 输出耦合度 C_e : 这个类中依赖于这个类的外部的类的数量
5. 继承树的深度：DIT = 从节点到根的继承树
6. 子类的数量：NOC = 一个类的直接子类的数量
7. 类凝聚力 LCOM

20. 封装

1. 封装将数据和行为同时包含在类中，分离对外接口与内部是实现。

20.1. 封装实现的细节

1. 封装数据和行为：Getter-Setter, 不是单单将方法和类成员关联
 - `getPositionArray()` 不好，应该是 `getPostion()`
2. 封装内部结构：迭代器模式
3. 封装其他对象的引用：不要返回内部的对象，新建一个返回
4. 封装类型信息：LSP
5. 封装潜在变更

21. 可修改性、可扩展性、灵活性

1. 实现的**可修改性**: 涉及到大的场景的修改

- i. 对已有实现的修改
- ii. 例如：修改现有促销策略
- 2. 实现的**可扩展性**(DIP & OCP)
 - i. 对新的实现的扩展
 - ii. 例如：增加一条新的促销策略(策略模式)
- 3. 实现的**灵活性**
 - i. 对实现的动态配置
 - ii. 例如：动态修改更改某商品对应促销策略

22. 几个设计模式

- 设计原则

- 接口重用
- 职责抽象
- 依赖倒置

- 设计模式

- 策略模式
- 抽象工厂模式
- 单键模式
- 迭代器模式

1. 策略模式:减少耦合、依赖倒置。
2. 抽象工厂模式:职责抽象、接口重用。
3. 单件模式:信息隐藏、职责抽象。
4. 迭代器模式:减少耦合、依赖倒置。

22.1. 策略模式

1. 在策略模式中，一个类的行为或其算法都可以在运行时改变。属**行为型模式**。
2. 在策略模式中，我们创建表示各种策略的对象和一个行为随着策略对象改变而改变的context对象。策略独享改变context对象的执行算法。

22.2. 抽象工厂模式

22.3. 迭代器模式

22.4. 单件模式

23. 软件构造

1. 软件构造除了核心的编程任务之外，还涉及详细设计(数据结构与算法设计)、单元测试、集成与集成测试、以及其他活动。
2. 重构：修改软件系统的严谨方法，它在不改变代码外部表现的情况下改进其内部结构。
3. 测试驱动开发：测试驱动开发要求程序员在编写一段代码之前，优先完成该段代码的测试代码。并在编程中重复执行测试代码，以验证程序的正确性。
4. 结对编程：两个程序员挨着坐在一起，共同协作进行软件构造活动。

24. 代码改进

1. 简洁性/可维护性
2. 使用数据结构消减复杂判定
3. 控制结构
4. 变量使用
5. 语句处理
6. How to write unmaintainable code
7. 防御与错误处理

25. 测试

25.1. 单元测试构造

1. 按照输入输出测试构造

- 为方法开发测试用例主要使用两种线索：
 - (1) 方法的规格；
 - 根据第一种线索，可以使用基于规格的测试技术开发测试用例，等价类划分和边界值分析是开发单元测试用例常用的黑盒测试方法。
 - (2) 方法代码的逻辑结构。
 - 根据第二种线索，可以使用基于代码的测试技术开发测试用例，对关键、复杂的代码使用路径覆盖，对复杂代码使用分支覆盖，简单情况使用语句覆盖。
- 在复杂类中，常常有着多变的状态，每次一个方法的执行改变了类状态时，都会给其他方法带来影响，也就是说复杂类的多个方法间是互相依赖的。
- 所以，除了测试类的每一个方法之外，还要测试类不同方法之间的互相影响情况。

2. 单元测试用例：输入，方法，预期输出

26. 契约式编程

1. 使用异常或断言的方式检查
2. 检查中间的过程

27. 防御式编程

1. 前置和后置方法

- 防御式编程的基本思想是：在一个方法与其他方法、操作系统、硬件等外界环境交互时，不能确保外界都是正确的，所以要在外界发生错误时，保护方法内部不受损害。
- 常见场景
 - 输入参数是否合法？
 - 用户输入是否有效？
 - 外部文件是否存在？
 - 对其他对象的引用是否为NULL？
 - 其他对象是否已初始化？
 - 其他对象的某个方法是否已执行？
 - 其他对象的返回值是否正确？
 - 数据库系统连接是否正常？
 - 网络连接是否正常？
 - 网络接收的信息是否有效？
- 异常和断言都可以用来实现防御式编程，两种实现方式的差异与契约式设计的实现一样。

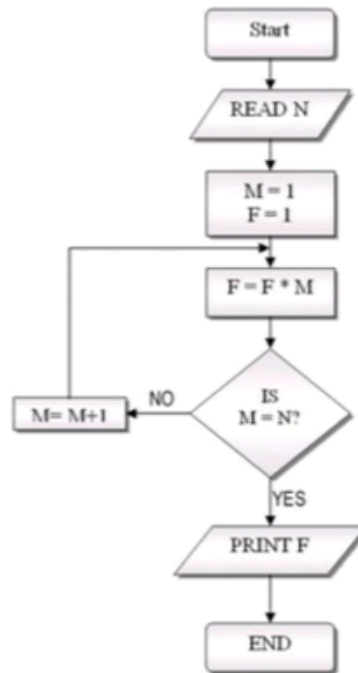
28. 表驱动

1. 决策表辅助

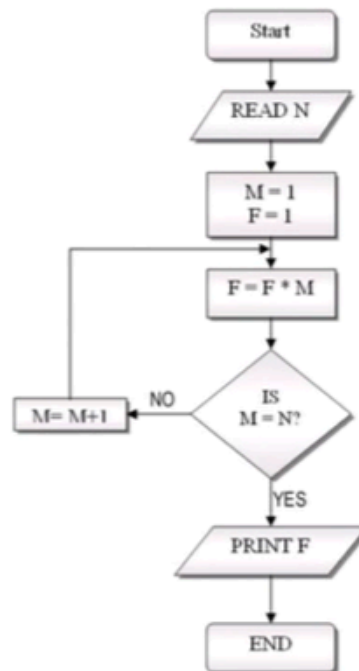
29. 代码复杂度度量

- 程序复杂度是造成各种编程困难的主要原因。
[Dijkstra 1972]很早就指出：“有能力的程序员会充分认识到自己的大脑容量是多么地有限；所以，他会非常谦卑的处理编程任务。”
- 为了帮助程序员处理程序复杂度，人们提出了很多程序复杂度的度量手段，其中McCabe的圈复杂度[McCabe 1976]得到了比较大的关注。
- 重要:这里涉及到两种计算方法

- 衡量圈复杂度的基本思路是计算程序中独立路径的最大数量。第一种计算方法是建立程序的流程图G，假设图的节点数为N，边数为E，那么复杂度 $V(G) = E - N + 2$ 。以右图为例，其节点数为8，边数为8，则程序的复杂度为2。通过直接分析右图也可以发现，它的确有两条路径。



- 还有一种简单的算法是直接计数程序中决策点的数量：
(1) 从1开始，一直往下通过程序。
(2) 一旦遇到下列关键字，或者同类的词，就加1：if, while, repeat, for。
(3) 给case语句中的每一种情况都加1。
- 例如，右图所描述的程序很明显只有一个DO-While语句，所以复杂度为2。



30. 白盒测试和黑盒测试

30.1. 黑盒测试

1. 黑盒测试是将测试对象作为一个黑盒子，完全基于输入和输出数据来判断测试对象的正确性。

30.1.1. 黑盒测试方法-等价类划分

黑盒测试方法 - 等价类划分

有效等价类：是指对于程序的规格说明来合理、意义输入数据构成集合。利用有效等价类可检验程序是否实现了规格说明中所定的功能和性能。

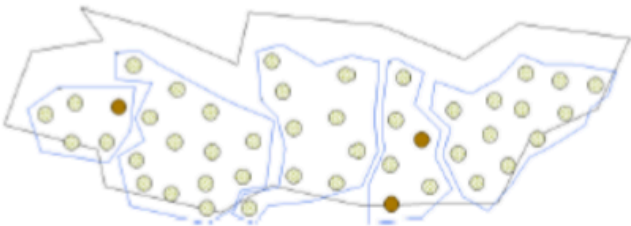


图 19-8 等价类划分

- 把所有可能的输入数据，即程序的输入域划分为若干部分(子集)，从每个子集中选取少量具有代表性的数据作为测试用例。
- 例子

Sale类 getChange(double payment)方法

所示的规格说明，可以将其输入数据划分为三类：

- 1有效数据；
- 2无效数据， $payment \leq 0$
- 3无效数据， $payment < total$

表 19-3 Sales 类 getChange(double payment) 方法的等价类划分测试用例

ID	输入		预期输出
1	payment=100	Total=50	50
2	payment=-100	total=20	输入数据无效
3	payment=50	total=100	输入数据无效

30.1.2. 黑盒测试方法-边界值分析

1. 边界值分析是等价类划分方法的补充，实践证明，错误最容易发生在各等价类的边界，而不是等价类的内部。
2. 对定价类划分的补充，错误容易发生在各个等价类的边界上，而不是等价类的内部，因此针对边界情况设计测试用例，可以发现更多的缺陷。

表 19-4 Sales 类 getChange(double payment) 方法的边界值分析测试用例

ID	输入		预期输出
1	payment=50	total=50	0
2	payment=50	total=49	1
3	payment=50	total=51	输入数据无效
4	payment=1	total=1	0
5	payment=0	total=0	0
6	payment=-1	total=10	输入数据无效

30.1.3. 黑盒测试方法-决策表

- 决策表是为复杂逻辑判断设计测试用例的技术。决策表示由条件声明、行动声明、规则选项和行动选项等四个象限组成的表格。

- ①prePoint<1000&&postPoint>=1000 → GiftLevel=1;
- ②prePoint<2000&&postPoint>=2000 → GiftLevel=2;
- ③prePoint<5000&&postPoint>=5000 → GiftLevel=3。

表 19-5 礼品赠送事件的测试用例

ID	输入		预期输出
1	prePoint=500	postPoint=1500	GiftLevel=1
2	prePoint=500	postPoint=2500	GiftLevel=2
3	prePoint=500	postPoint=5500	GiftLevel=3

30.1.4. 黑盒测试方法-状态转换

- 状态转换是针对复杂测试对象的测试技术。该类复杂测试对象对输入数据的反映是多样的，还需要依赖自身的状态才能决定。
- 状态转换包含有效转换和无效转换，只有在复杂情况和可靠性要求较高的情况下才会为无效转换设计测试用例。

- 使用状态转换测试技术时，通常要先为对象建立状态图，描述测试对象的状态集合、输入集合和输入导致的状态转换集合。
- 以状态图基础，可建立测试对象的转换表。状态转换表每一行都应该被设计为测试用例。

- 例子

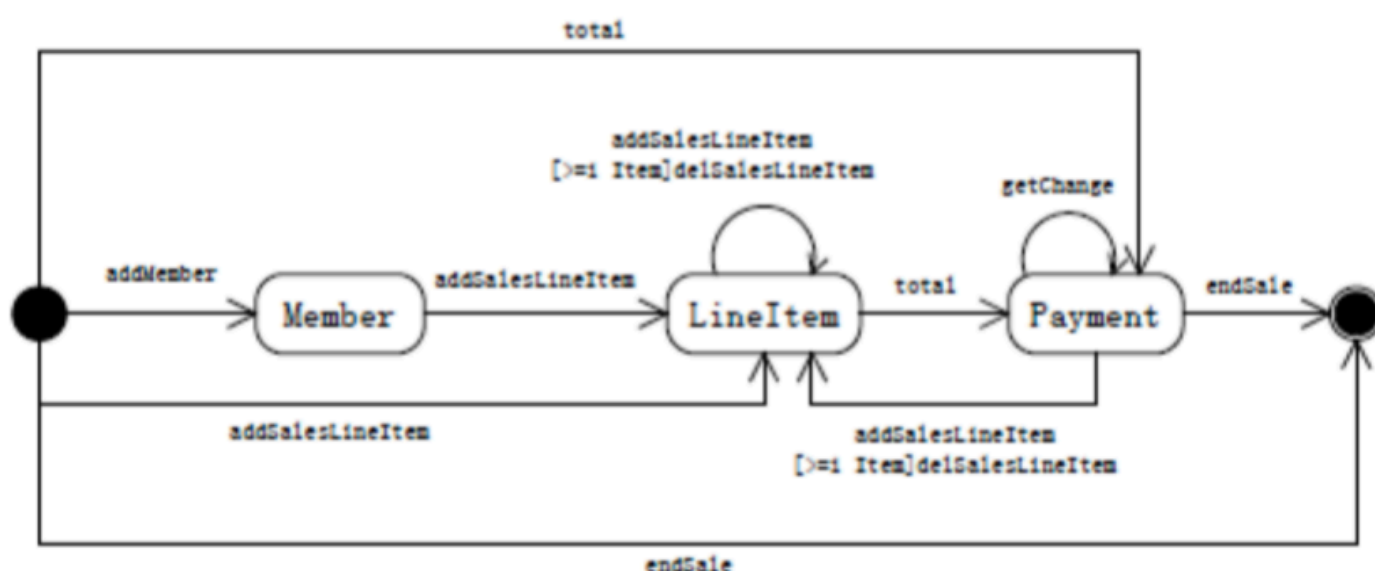


表 18-4 类 Sales 的测试用例线索

输入		预期输出状态
方法	当前状态	
addMember	Start	Member
	Member	非法
	LineItem	非法
	Payment	非法
	End	非法
addSalesLineItem	Start	LineItem
	Member	LineItem
	LineItem	LineItem
	Payment	LineItem
	End	非法
[>=1 Item] delSalesLineItem		
total		
getChange		
endSale		

表 18-5 类 Sales 的测试用例

ID	输入		预期输出
	前置语句	方法	
1	s=new Sales();	s.addMember(2);	No Exception
2	s=new Sales(); s.addMember(1);		MemberLable Invalid Time
3	s=new Sales(); s.addSalesLineItem(1);		
4	s=new Sales(); s.addSalesLineItem(1); s.total();		Sales dose not Exists
5	s=new Sales(); s.addSalesLineItem(1); s.total(); s.getChange(100); s.endSale();		

30.2. 白盒测试

1. 白盒测试是将测试对象看做透明的，不关心测试对象的规格，而是按照测试对象内部的程序结构来设计测试用例进行测试工作。

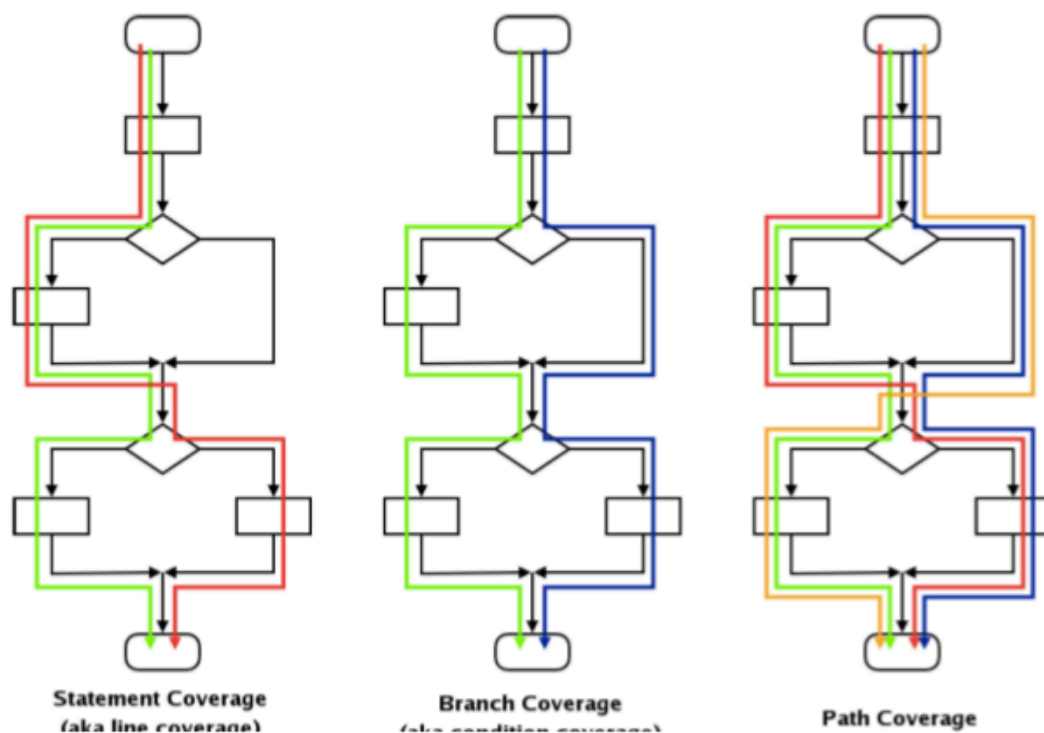
30.2.1. 白盒测试方法-语句覆盖

1. 语句覆盖设计测试用例的标注是保证被测试的对象的每一行代码都至少执行了一次。

白盒测试方法 - 语句覆盖

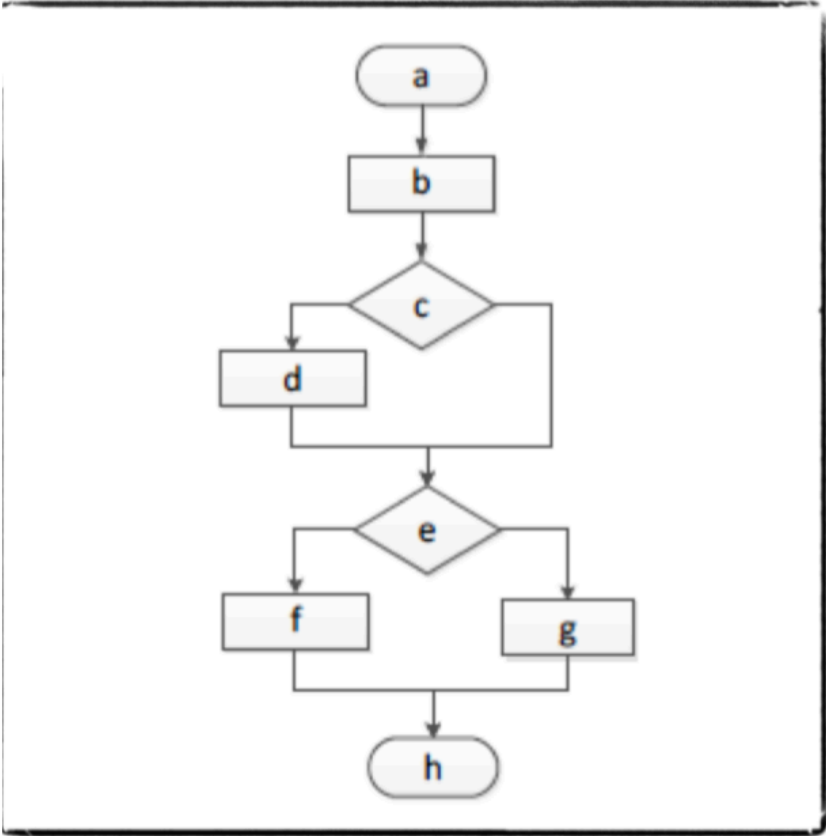
语句覆盖设计测试用例的标准是确保被测试对象的每一行程序代码都至少执行一次。

```
public Class Customer {  
    int bonus;//积分额度  
    .....  
    //预计算消费行为后的积分额  
    int getBonus ( boolean cashPayment , int consumption , bo  
        //已有的 bonus 需要调整 getBonus，不能直接使用  
        int preBonus=this.getBonus();  
        if (cashPayment) {  
            preBonus+=consumption;  
        }  
        if (vip) {  
            preBonus*=1.5;  
        } else {  
            preBonus*=1.2;  
        }  
        return preBonus;  
    }  
}
```

白盒测试

- 所有的路径都覆盖一遍，路径覆盖



为了清晰地解释对语句的覆盖，图19-10为各条语句都进行了编号。可以发现如果按照下列两条路径执行，就能够覆盖所有的语句：a,b,c,d,e,f,h 和 a,b,c,d,e,g,h

30.2.2. 白盒测试方法-条件覆盖

1. 条件覆盖设计测试用例的标准是确保程序中每个判断的每个结果都至少满足一次。

白盒测试方法 - 条件覆盖

条件覆盖设计测试用例的标准是确保程序中每个判断的每个结果都至少满足一次

表 19-7 Customer.getBonus() 的条件覆盖测试用例

ID	输入				预期输出
1	getBonus ()=100	cashPayment=true	consumption=100	vip=true	300
2	getBonus ()=100	cashPayment=false	consumption=100	vip=false	120

30.2.3. 白盒测试方法-路径覆盖

1. 路径覆盖测试用例的标准是确保程序中每一条独立执行的路径都至少执行一次。

31. 测试

- 对给定的场景和要求的测试方法，设计测试用例
 - 给出功能需求，则要求写功能测试用例
 - 给出设计图，则要求写集成测试用例，Stub and Driver
 - 给出方法的描述，则要求写单元测试用例，Mock Object
- JUnit基本使用方法
 - 单元测试用例
 - 功能测试用例：完整有价值的序列
 - 测试的度量
 - 需求覆盖率=被测试的需求数量/需求总数
 - 模块覆盖率=被测试的模块数量/模块总数
 - 代码覆盖率=被测试的代码行/代码行数

32. 软件维护

1. 软件维护是在交付之后修改软件系统或其部件的活动过程，以修正缺陷、提高性能或其他属性、适应变化的环境。

32.1. 软件维护的重要性

1. 主要是为了保证产品的正常运转而进行使用帮助、故障解决和磨损处理等工作。
2. 软件维护是在交付之后修改软件系统或其部件的活动过程，以修正缺陷、提高性能或其他属性、适应变化的环境。问题、环境发生了改变，软件产品中存在缺陷。
3. 软件的维护主要是"修改"。软件不会磨损，软件维护只需要完成少量的使用帮助、故障解决和磨损处理等工作，软件特性上是易于修改的，软件只有持续修改才能保持价值。

32.2. 开发可维护软件的方法

1. 前期(开发阶段)更充分的准备可以减轻后期维护的压力与困难
2. 考虑到软件的可变更性进行开发

32.3. 软件维护的过程

1. 问题/修改的标识、分类与划分优先级
2. 分析
3. 设计
4. 实现
5. 回归测试：修改后不会影响
6. 验收测试
7. 移交

32.4. 用户文档和系统文档

32.4.1. 用户文档

1. 用户文档是指为用户编写参考指南或者操作教程，常见的如用户使用手册、联机帮助文档等，统称为用户文档。
2. 文档内容的组织应当支持其使用模式，常见的是指导模式和参考模式两种。

32.4.2. 用户文档的要素

1. 标识信息
 - i. 包括文档标题、文档产生的版本和日期、相关的软件 产品和版本
 - ii. 标识信息应该放在包装袋或者封面，用户可以不用翻阅文档就能看到
2. 引言
 - i. 正文的第一部分，描述文档的预期读者、描述范围，以及对文档目的、功能和操作环境的概要描述
3. 文档使用信息
 - i. 文档使用信息描述了关于文档的使用信息，例如，解释各种图示的含义、介绍如何使用帮助等。
4. 操作模式
 - i. 操作模式是使用用户文档的模式，例如对操作流程的图示或者文字性描述，再例如解释操作的理论、原因、算法或者通用概念。
5. 操作规程
 - i. 指导模式文档应该包括很多软件功能都会涉及的常见活动规程：
 - a. 需要由用户执行的软件安装与卸载
 - b. 图形用户界面特性的使用指导
 - c. 访问、登录或者关闭软件
 - d. 通过软件的导航，访问和退出相关功能
 - e. 数据操作(输入、保存、读取、打印、更新和删除)
 - f. 取消、中断和重启操作的方法
 - ii. 对于完成用户任务的操作流程，指导模式文档应该从基本信息、指导步骤和结束信息三个方面来描述
 - iii. 基本信息：
 - a. 简要概述操作规程的目的，定义或解释必要的概念
 - b. 标明执行任务前需要完成的技术活动
 - c. 列举用户完成任务所需要的资源情况，例如数据、文档、密码等
 - iv. 对于完成用户任务的流程，指导模式文档应该从基本信息、指导步骤和结束信息三个方面来描述
 - v. 指导步骤：使用祈使语句描述用户行为，并指出预期的结果。指导步骤要说明：
 - a. 用户输入数据的域值范围、最大长度和格式
 - b. 相应的错误消息和恢复办法

c. 其它可选的步骤和重复步骤

vi. 对于完成用户任务的操作流程，指导模式文档应该从基本信息、指导步骤和结束信息三个方面来描述

vii. 结束信息：标明操作规程的最后步骤，让用户知道怎样判断整个操作规程的成功完成，告诉用户如何退出操作规程

6. 软件命令信息

i. 解释用户输入命令的格式和操作规程，包括必要参数、可选参数、缺省值等，要示例说明命令的使用，说明怎样判断命令是成功完成还是异常中止

7. 错误信息与问题解决

i. 文档要详细描述软件使用中的已知问题，让用户清楚如何自行解决问题或者怎样向技术支持人员报告准确的信息

8. 导航特征

i. 包括章节、主题、页码、链接、图标等

ii. 提高导航特征和效率

32.4.3. 系统文档

1. 与用户文档注重系统使用细节不同，系统管理员文档更注重系统维护方面的内容，例如系统性能调整、访问权限控制、常见故障解决等等。因此，系统管理员文档需要详细介绍软硬件的配置方式、网络连接方式、安全验证与访问授权方法、备份与容灾方法、部件替换方法等等。

32.5. 软件演化生命周期模型

1. 人们将软件从**生产到报废**的生命周期分割为不同阶段，每段阶段有明确的典型输入/输出、主要活动和执行人，各个阶段形成明确、连续的顺次过程，这些阶段划分就被称为软件生命周期模型。

32.5.1. 步骤

1. 初始开发
2. 演化
3. 服务
4. 逐步淘汰
5. 停止

32.5.2. 优缺点

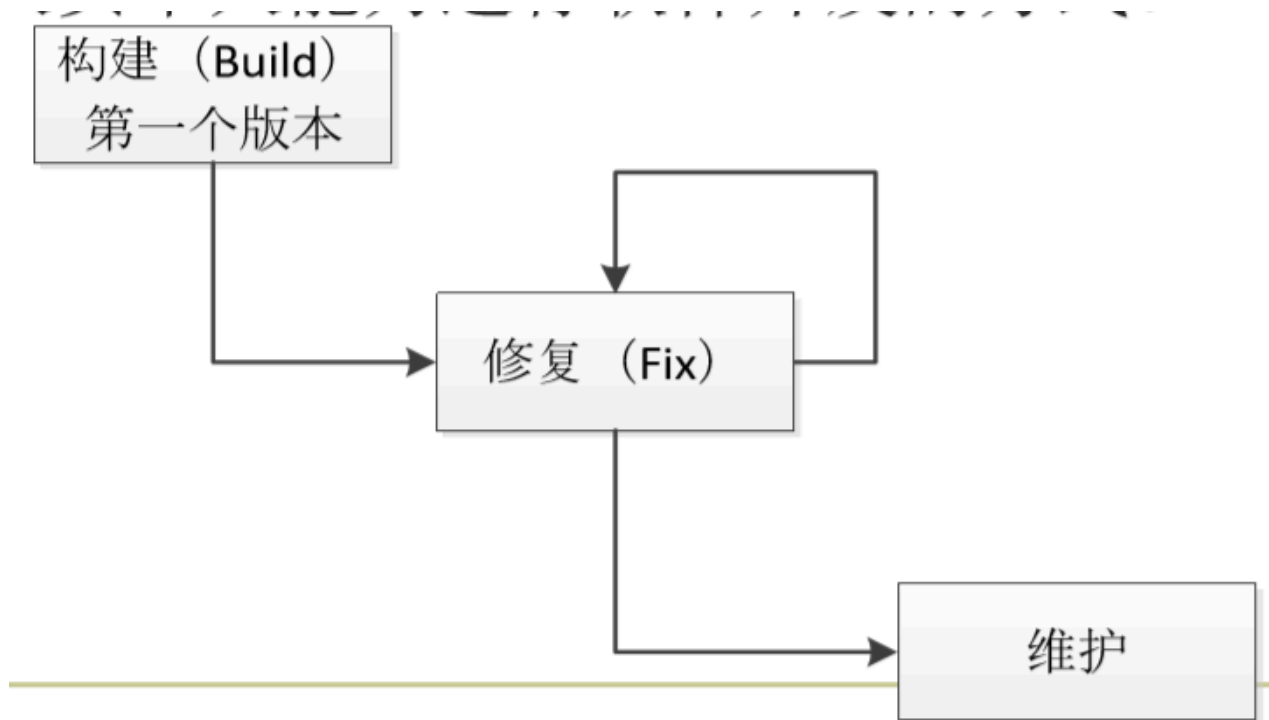
1. 优点：

- 使用了迭代式开发，具有更好的适用性、尤其是其演化式迭代安排能够适用于那些需求变更比较频繁或不确定性较多的软件系统的开发
- 并行开发可以帮助缩短软件产品的开发时间
- 渐进交互可以加强用户反馈，降低开发风险

2. 缺点：

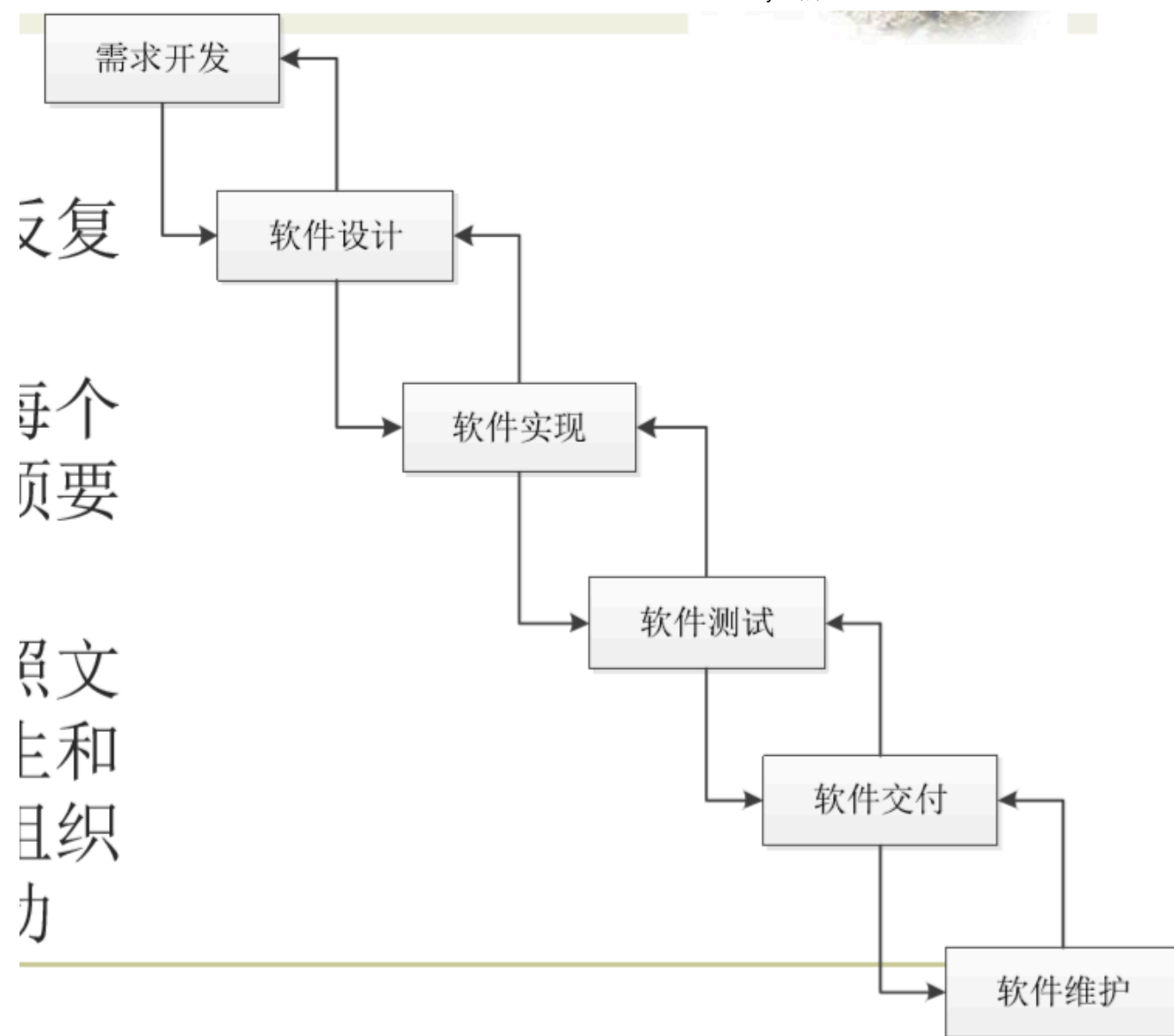
- 无法在项目早期阶段确定项目范围，所以项目的整体计划、进度调度、尤其是商务协商事宜无法准确把握
- 后续迭代的开发活动是在前导迭代的基础上进行修改和扩展的，这容易让后续迭代忽略分析与设计工作，蜕变为构建——修复方式
- 如果缺乏过程管理，可能会陷入试——错——改的循环

32.5.3. 实际模型



1. 构建-修补模型

- i. 最早也是最自然产生的软件开发模型，对软件开发活动没有任何规划和组织，是完全依靠开发人员个人能力进行软件开发的方式。
- ii. 缺点：
 - a. 在这种模型中，没有对开发工作进行规范和组织，所以随着软件系统的**复杂度提升**，开发活动会超出个人的直接控制能力，构建-修复模型就会导致开发活动无法有效进行而失败；
 - b. 没有分析**需求的真实性**，给软件开发带来很大的风险；
 - c. 没有考虑**软件结构的质量**，使得软件结构在不断的修改中变得质量越来越糟，直至无法修改；
 - d. 没有考虑测试和程序的可维护性，也没有任何文档，软件的维护十分困难。
- iii. 适用：软件规模很小的软件



2. 瀑布模型

- i. 瀑布模型是按照软件生命周期模型将软件开发活动组织为需求开发、软件设计、软件实现、软件测试、软件交付和软件维护等活动，并且规定了它们自上而下、相互邻接的次序。
- ii. 优点：为软件开发活动定义了清晰的阶段划分(包括输入/输出、主要工作及其关注点)，这让开发者能够以关注点分离的方式更好地进行那些复杂度超越个人能力的软件项目的开发活动。
- iii. 缺点：
 - a. 对文档的过高的期望具有局限性
 - b. 对开发活动的线性顺序假设具有局限性
 - c. 客户、用户的参与具有局限性：成功的项目开发需要客户、用户从始至终的参与，而不仅仅是一个阶段。
 - d. 里程碑粒度具有局限性：里程碑粒度过粗，基本丧失了"早发现缺陷早修复"这一思想
- iv. 适用：
 - a. 适用于比较成熟，没有技术难点的软件



3. 增量迭代模型(需求驱动)

- i. 增量迭代模型是在项目开始时，通过系统需求开发和核心体系结构设计活动完成项目对前景和范围的界定，然后再将后续开发活动组织为多个迭代、并行的瀑布式开发模型。需求驱动。
- ii. 少量的不确定性和影响不大的需求变更通过迭代的方式加以解决
- iii. 优点:
 - a. 迭代式开发更加符合软件开发的实践情况，具有更好的适用性；
 - b. 并行开发可以帮助缩短软件产品的开发时间；
 - c. 渐进交付可以加强用户反馈，降低开发风险。
- iv. 缺点:
 - a. 由于各个构件是逐渐并入已有的软件体系结构中的，所以加入构件必须不破坏已构造好的系统部分，这需要软件具备开放式的体系结构。
 - b. 增量交付模型需要一个完备、清晰的项目前景和范围以进行并发开发规划，但是在一些不稳定的领域，不确定性太多或者需求变化非常频繁，很难在项目开始就确定前景和范围。
- v. 适用范围：适用于大规模软件系统的开发



4. 演化模型

- i. 演化模型将软件开发活动组织为多个迭代、并行的瀑布式开发活动。演化模型能够更好地应对需求变更，更适用于需求变更比较频繁或不确定性较多的领域
- ii. 优点:
 - a. **使用了迭代式开发，具有更好的适用性**，尤其是其演化式迭代安排能够适用于那些需求变更比较频繁或不确定性较多的软件系统的开发；
 - b. 并行开发可以帮助**缩短**软件产品的开发时间；
 - c. 渐进交付可以**加强用户反馈**，降低开发风险。
- iii. 缺点:
 - a. 无法在项目早期阶段建立项目范围，所以项目的整体计划、进度调度、尤其是商务协商事宜无法准确把握；
 - b. 后续迭代的开发活动是在前导迭代基础上进行修改和扩展的，这容易让后续迭代忽略设分析与设计工作，蜕变为构建-修复方式。
- iv. 适用与不稳定领域的大规模软件系统开发



5. 原型模型

i. 原型模型将需求开发活动展开为抛弃式原型开发活动和演化式原型开发活动。原型模型在整体安排迭代的情况下，强调“**抛弃式原型**”的演化模型。抛弃式原型解决对未来知识的局限性产生的**不确定性**，将未来置于现在进行推敲。

ii. 抛弃式原型

- 它通过模拟“未来”的产品，将“未来”的知识置于“现在”进行推敲，解决不确定性。
- 存在的原因是“不确定的”，这一类原型在后续的开发过程中会被抛弃

iii. 演化式原型

- 在迭代中构建，是系统的核心，并不断扩充，最终成为真正的软件产品。
- 它将作为真正产品的一部分，所以必须有很好的质量。在迭代式开发中，通常会在第一个迭代中构建一个核心的体系结构演化式原型，并且在后续迭代中不断扩充，成为真正的软件产品。

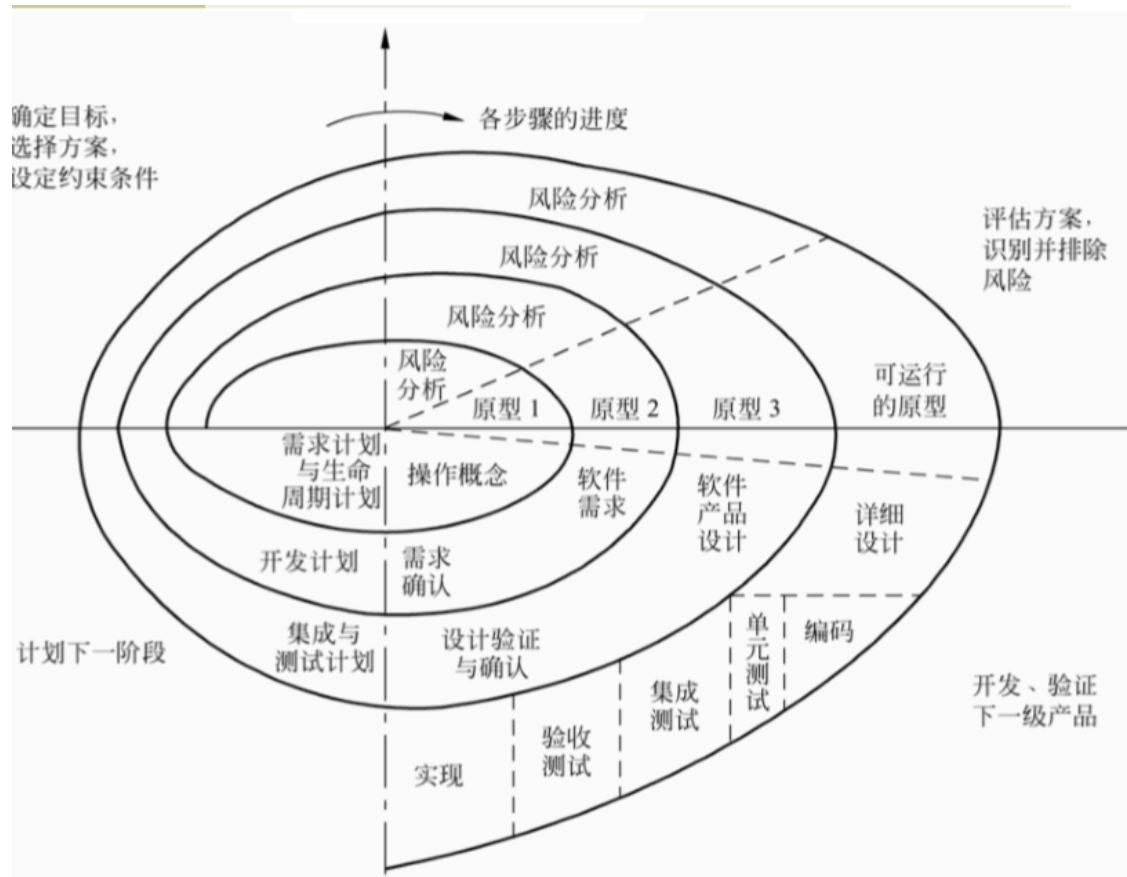
iv. 优点：

- 对原型方法的使用加强了与客户、用户的交流，可以让最终产品取得更好的满意度；
- 适用于非常新颖的领域，这些领域因为新颖所以有着大量的不确定性。

v. 缺点

- 原型方法能够解决风险，但是自身也能带来新的风险，例如原型开发的成本较高，可能会耗尽项目的费用和时间；
- 实践中，很多项目负责人不舍得抛弃“抛弃式原型”，使得质量较差的代码进入了最终产品，导致了最终产品的低质量。

vi. 适用性：适用于具有大量不确定的新颖领域进行开发活动组织。

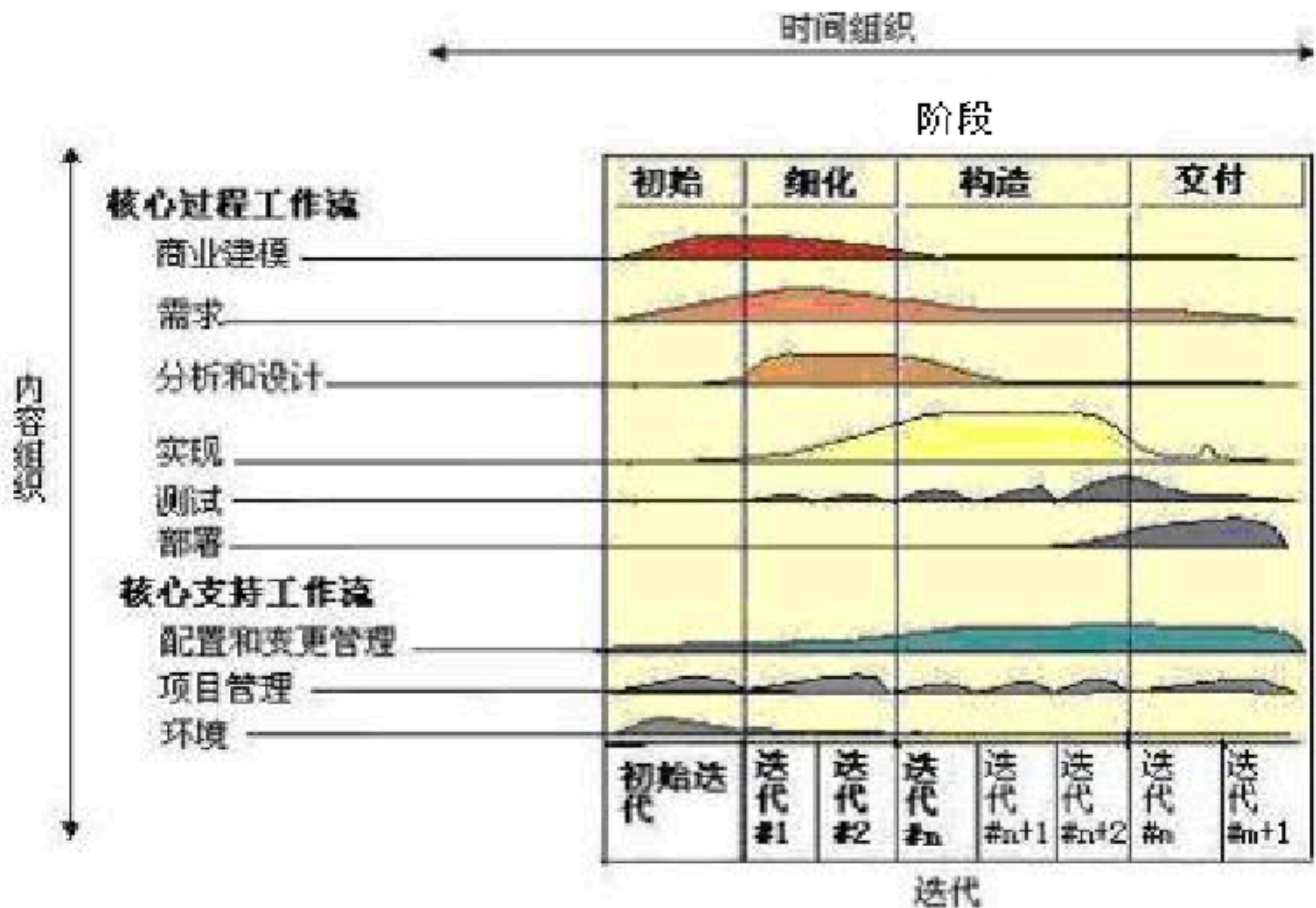


迭代与瀑布的结合

- 开发阶段是瀑布式的
- 风险分析是迭代的

6. 螺旋模型

- 螺旋模型是风险驱动的，完全按照风险解决的方式组织软件开发活动。
 - 确定目标、解决方案和约束
 - 评估方案，发现风险
 - 寻找风险解决方法
 - 落实风险解决方案
 - 计划下一个迭代
- 自内向外，螺旋模型有4次风险解决迭代，分别解决了几个高风险的阶段的问题。
 - 解决系统需求开发中的风险，尤其是产品概念设计风险，得到一个确定的产品前景和范围。
 - 解决软件需求开发中的风险，得到清晰的软件需求
 - 解决软件体系结构设计中的技术风险，构建高质量的核心体系结构原型。
 - 解决详细设计和实现中的关键技术风险，建立一个可实现的高质量软件结构。
- 优点：可以降低风险，减少项目因风险造成的损失
- 缺点：
 - 风险解决需要使用原型手段，也就会存在原型自身带来的风险，这一点与原型模型相同；
 - 模型过于复杂，不利于管理者依据其组织软件开发活动
- 适用于高风险的大规模软件系统开发



7. RUP模型

- 统一过程(Rational Unified Process, RUP)总结和借鉴传统上的各种有效经验，建立最佳实践方法的集合，并提供有效的过程定制手段，允许开发者根据特定的需要定制一个有效的过程模型。
- 实践方法：
 - 迭代式开发，这是过去被反复证明的最佳实践方法；
 - 管理需求，重视需求工程中除了需求开发之外的需求管理活动；
 - 使用基于组件的体系结构，它帮助建立一个可维护、易开发、易复用的软件体系结构；
 - 可视化建模，利用UML进行建模；
 - 验证软件质量，尽早和持续地开展验证，以尽早发现缺陷，降低风险和成本；
 - 控制软件变更，适应1990s以后需求变更越来越重要的事实。
- 优点：
 - 吸收和借鉴了传统上的最佳实践方法，尤其是其核心的6个实践方法，能够保证软件开发过程的组织是基本有效和合理的。
 - RUP依据其定制机制的不同，可以适用于小型项目，也可以适用于大型项目的开发，适用面广泛。
 - RUP有一套软件工程工具的支持，这可以帮助RUP的有效实施。
- 缺点：
 - 没有考虑交付之后的软件维护问题
 - 裁剪和配置工作不是一个简单的任务，无法保证每个项目都能定制一个有效的RUP过程。
- 适用：RUP是重量级过程，能够胜任大型软件团队开发大型项目时的活动组织。但RUP经过裁剪和定制，也可以变为轻量级过程，也能够胜任小团队的开发活动组织。

8. 敏捷模型(XP)

9. 最为重要的敏捷思想是敏捷联盟宣言所声明的价值观：

- 个体和互动 高于 流程和工具
- 工作的软件 高于 详尽的文档
- 客户合作 高于 合同谈判

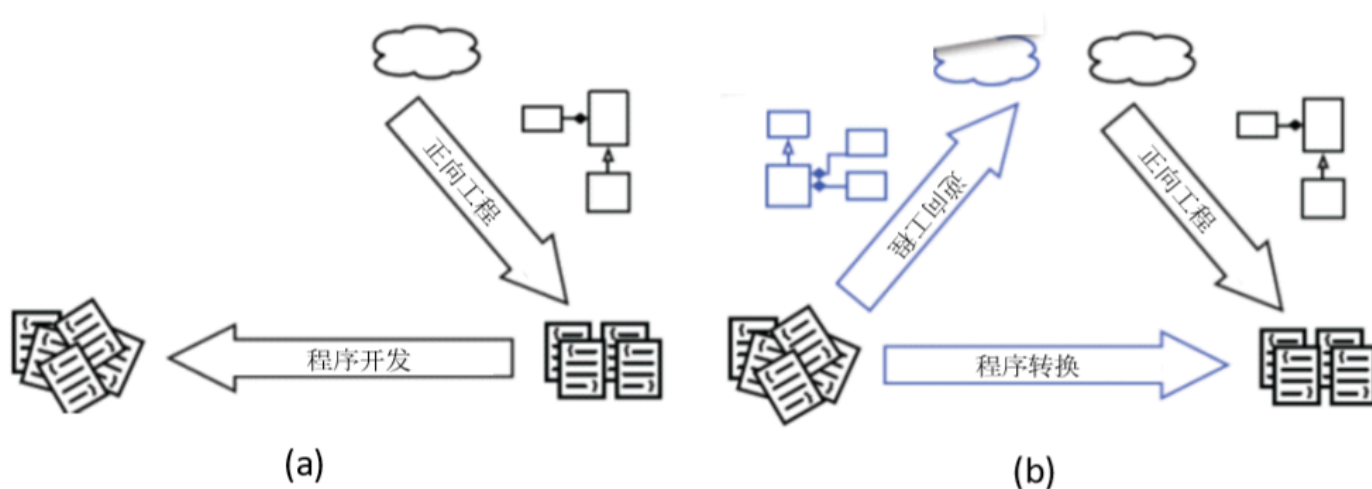
iv. 响应变化 高于 遵循计划

10. 极限编程，就是什么最有效就做什么

11. 适用于快速变化或者时间压力较大的项目。

32.6. 逆向工程

1. 处理遗留软件时，维护人员接受的维护对象可能是一个没有任何文档也没有程序源代码的软件程序，此时，维护人员需要使用逆向工程技术
2. 逆向工程技术是指："分析目标系统，标识系统的部件及其交互关系，并且使用其它形式或者更高层的抽象创建系统表现的过程[Chikofsky1990]"。
3. 逆向工程的基本原理是抽取软件系统的需求与设计而隐藏实现细节，然后在需求与设计的层次上描述软件系统，以建立对系统更加准确和清晰的理解。



32.7. 再工程

1. [Arnold1993]认为再工程主要是下列两类活动：
 - i. 改进人们对软件的理解
 - ii. 改进软件自身，通常是提高其可维护性、可复用性和可演化性
2. 常见的具体活动有
 - i. 重新文档化
 - ii. 重组系统的结构
 - iii. 将系统转换为更新的编程语言
 - iv. 修改数据的结构组织。
3. 是抽象出来一个视图，然后完全重新开始开发，而不是基于逆向工程得到的结果进行再工程

33. 部分题目

1. 为了有效地捕获系统需求,应采用(C) 。
- A.瀑布模型
- B.V 模型
- C.原型模型
- D.螺旋模型
2. 软件开发过程中, 需求分析阶段的输出不包括(D)
- A.数据流图
- B.实体联系图
- C.数据字典
- D.软件体系结构图
3. "软件产品必须能够在 3 秒内对用户请求作出响应"属于软件需求中的(B)
- A.功能需求
- B.非功能需求
- C.设计约束
- D.逻辑需求
4. 按照功能分解的方式进行模块分割能够实现高内聚的软件设计: ✓
5. 软件系统设计的主要目的是为系统制定蓝图, (D)并不是软件设计模型所关注的。
1. 系统总体结构
2. 数据结构
3. 界面模型
4. 项目范围(这个是在需求部分已经完成)
6. 体系结构设计是软件非功能性的实现, 而详细设计主要是软件功能性的实现。✓

34. 说明以下需求属于哪种类型

需求描述	需求类别	备注
系统 需要 存储一年内的销售记录	数据需求	-
系统 应该能够 存储3年的交易数据	性能需求	-
系统使用之前, 需要对收银员进行10天的专门培训	其他需求 (包括硬件需求、人力需求。)	在交付之前
经过10天培训的收银员就能够熟练使用系统	软件需求 (中的质量需求)	-
反映了用户与系统的 交互细节	系统需求	-
反映了用户与系统的 交互	用户需求	-

需求描述	需求类别	备注
在存储设备发生故障时，系统要在10秒内发现	质量需求 (可靠性需求)	故障是非正常状态， 还一种系统需求。
在存储设备发生故障时，系统要在0.5秒内向用户发出 警报	功能需求	故障不是系统的正常状态
检测到病人异常后，监控器必须在0.5秒内发出警报	性能需求	-
该软件管理工具的开发过程 自身 必须符合CMMI-4的评估	过程需求	是软件自身
该软件管理工具软件必须 帮助 项目管理者进行开发管理工作， 以通过CMMI-4的评估	用户需求	-
系统能够为用户提供库存分析报告、商品/ 利润报告和过期商品报告	用户需求	-
产品在发布1年之后内，必须在出版的A、B、 C三个产品评论刊物中被评为最可靠产品。	业务需求	相当与一个目的
系统必须能够与Oracle数据库交互。	约束	Oracle是系统的一个环境， 环境本身在约束部分， 比如Tomcat等等
开发团队需要给出SRS文档。	过程需求	-
使用扫描仪扫描文件，传递回的数据为pdf格式 文件。	对外接口	软件和扫描仪之间的交互
商品的标识由0-24位字母、数字混合组成的字符串。	数据需求	-
商品标识的类型要能够在0.5个人月内更改为长整型。	质量需求	可移植性

1. 其他描述用例是需求的一种组织和表达形式。√

35. 判断需求的正确性或者问题

需求描述	需求正确与否	问题
After the payment process is complete, the relevant information should be appended to a log file.	错误	模糊
The system should be constructed so that it will be easy to add new functionality in the future.	错误	模糊
The price of a gasoline purchase is computed as the price per gallon for the type of gas purchased, multiplied by the number of gallons purchased (use two decimal points for representing fractions of gallons).	正确	-
The system should be available 24 hours a day, 7 days a week.	错误	不切实际

1. 对象之间的关系：系统顺序图